

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

федеральное государственное бюджетное образовательное учреждение
высшего образования
«УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет Информационных систем и технологий

Кафедра Вычислительная техника

К ЗАЩИТЕ ДОПУСТИТЬ

Зав. кафедрой

_____ / К.В. Святков /
подпись инициалы, фамилия

«___» _____ 2026 г.

Бакалаврская работа

Вид ВКР (дипломный проект (работа) / бакалаврская работа / магистерская диссертация)

Тема «Разработка фреймворка автоматизации UI-тестирования веб-приложений на Java»

Обучающийся _____ / И.Е. Борков /
подпись инициалы, фамилия

Обозначение ВКР БР-УлГТУ-09.03.01-22/2302-2026 Группа ИВТАСбд-41
для технических направлений подготовки (специальностей)

Направление подготовки (специальность) 09.03.01 «Информатика и
код, наименование
вычислительная техника», профиль «Автоматизированные системы
обработки
информации и управления»

Руководитель ВКР _____ / В.В. Валюх /
подпись, дата инициалы, фамилия

Ульяновск
2026 г.

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

федеральное государственное бюджетное образовательное учреждение

высшего образования

«УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет Информационных систем и технологий

Кафедра Вычислительная техника

УТВЕРЖДАЮ

Зав. кафедрой

_____ / К.В. Святлов /
подпись инициалы, фамилия

« » 2026 г.

ЗАДАНИЕ

На _____ выпускную работу бакалавра

(наименование выпускной квалификационной работы)

обучающемуся Боркову Ивану Евгеньевичу курса 4 группы ИВТАСбд-41

фамилия, имя, отчество

Тема ВКР «Разработка фреймворка автоматизации UI-тестирования веб-приложений на Java»

утверждена приказом по УлГТУ от «19» декабря 2025 г. № 3379

Срок сдачи обучающимся законченной ВКР «5» июня 2026 г.

Исходные данные к ВКР

- 1) В ходе выполнения работы должен быть произведён анализ существующих средств и технологий автоматизации тестирования веб-приложений;
- 2) Разрабатываемая система должна реализовывать следующие функциональные возможности:
 - Автоматизированное выполнение UI-тестов веб-приложения;
 - Поддержку параллельного выполнения тестов;
 - Распределённый запуск тестов с использованием Selenium Grid;
 - Контейнеризацию тестовой инфраструктуры с использованием Docker;
 - Формирование отчётности о результатах тестирования;

Содержание пояснительной записки (перечень подлежащих разработке вопросов)

- Анализ предметной области автоматизированного тестирования;
- Анализ существующих средств автоматизации UI-тестирования;
- Проектирование архитектуры тестового фреймворка;
- Реализация фреймворка автоматизации тестирования;
- Организация распределённого и параллельного выполнения тестов;
- Тестирование разработанного фреймворка и анализ результатов.

Перечень графического материала (с точным указанием обязательных чертежей)

Диаграмма архитектуры тестового фреймворка, структура проекта автоматизированного тестирования, схема взаимодействия Selenium Grid, схема контейнеризированной инфраструктуры Docker, диаграмма процесса выполнения UI-тестов, результаты выполнения тестов и отчёты Allure Report.

Дата выдачи задания «19» декабря 2025 г.

Руководитель к.т.н. доцент кафедры «ВТ» /В.В. Валюх/
должность, учёная степень, ученое звание подпись инициалы, фамилия

Задание принял к исполнению _____ / И.Е. Борков /
подпись обучающегося инициалы, фамилия



АННОТАЦИЯ

**к бакалаврской работе Боркова Иван Евгеньевича на тему
«Разработка фреймворка автоматизации UI-тестирования веб-
приложений на Java»**

Кафедра ВТ УлГТУ, 2026г.

Научный руководитель: к.т.н. доцент кафедры «ВТ» Валюх В. В.

Работа содержит 103 страницы, включающих: 21 иллюстраций, 9 таблицы, 7 листинга, 5 приложений и список информационных источников из 46 наименований.

Ключевые слова: автоматизация тестирования, UI-тестирование, Selenium WebDriver, Selenium Grid, Docker, TestNG, Java, Allure Report, параллельное тестирование.

Представленная бакалаврская работа посвящена разработке фреймворка автоматизации UI-тестирования веб-приложений на языке Java с использованием Selenium WebDriver, Selenium Grid и Docker.

Пояснительная записка содержит следующие разделы:

- 1) Теоретические основы тестирования программного обеспечения. В данной главе рассматриваются понятия, цели и виды тестирования программного обеспечения, анализируются современные средства автоматизации тестирования, а также особенности UI-тестирования веб-приложений.
- 2) Анализ требований и проектирование фреймворка автоматизации тестирования. В разделе определяются требования к архитектуре тестового фреймворка, описывается структура проекта, принципы работы Selenium Grid и Docker, а также проектирование параллельного выполнения тестов.
- 3) Реализация фреймворка автоматизированного тестирования. Глава содержит описание реализации компонентов системы, включая управление WebDriver, Page Object Model, тестовые сценарии, настройку Selenium Grid и Docker Compose, а также интеграцию

системы отчётности Allure Report.

4) Тестирование разработанного фреймворка. В данном разделе приводятся результаты выполнения UI-тестов, анализируется работа параллельного выполнения тестов, оценивается эффективность использования Selenium Grid и Docker, а также рассматриваются результаты формирования отчётности.

Содержание

Перечень используемых сокращений, обозначений, терминов	9
Введение	11
Глава 1. Теоретические основы тестирования программного обеспечения	14
1. Веб-приложения и особенности их тестирования	14
1.1.1. Веб-интерфейсы и их роль в современных информационных системах	14
1.1.2. Тестирование программного обеспечения: понятие, цели и задачи	16
1.1.3. Виды тестирования программного обеспечения	18
1.1.4. Роль тестирования в разработке веб-интерфейсов	20
2. Автоматизированное тестирование программного обеспечения	23
1.2.1. Понятие и подходы к автоматизированному тестированию	23
1.2.2. Виды автоматизированного тестирования	25
1.2.3. Обзор средств автоматизированного тестирования	27
1.2.4. Преимущества и ограничения автоматизированного тестирования	30
3. Выбор средств и технологий для разработки автотестов	32
1.3.1. Критерии выбора инструментов автоматизации тестирования	32
1.3.2. Обоснование выбора стека для разработки автотестов	34
4. Выводы по первой главе	35
Глава 2. Теоретические основы тестирования программного обеспечения	38
1. Анализ требований к системе автоматизированного тестирования	38
2.1.1. Цели и задачи разрабатываемого фреймворка	38
2.1.2. Требования к архитектуре тестового фреймворка	39
2.1.3. Требования к архитектуре тестового фреймворка	42
2. Проектирование архитектуры фреймворка автоматизации тестирования	44
2.2.1. Выбор архитектурного подхода к построению фреймворка	44
2.2.2. Общая архитектура системы автоматизированного тестирования	48
2.2.3. Структура проекта автоматизированного тестирования	51
3. Проектирование распределённого выполнения тестов	53
2.3.1. Принципы работы Selenium Grid	53
2.3.2. Использование Docker для контейнеризации среды тестирования	55
2.3.3. Проектирование параллельного запуска тестов	56
4. Проектирование системы отчётности и интеграции	58

2.4.1. Организация формирования отчётов о тестировании	58
2.4.2. Сценарий работы разработанного фреймворка	60
5. Выводы по второй главе	63
2.5.1. Основные результаты проектирования	63
2.5.2. Подготовка к реализации фреймворка	64
Глава 3. Реализация фреймворка автоматизации UI-тестирования веб-приложений	66
1. Реализация архитектуры и инфраструктуры фреймворка	66
3.1.1. Настройка проекта и используемых технологий	66
3.1.2. Реализация архитектуры фреймворка	68
3.1.3. Реализация распределённой инфраструктуры тестирования	70
2. Реализация автоматизированных UI-тестов	73
3.2.1. Реализация тестовых сценариев	73
3.2.2. Реализация Page Object компонентов	75
3. Формирование отчётности и анализ результатов тестирования	77
3.3.1. Интеграция Allure Reports	77
3.3.2. Анализ результатов выполнения тестов	79
4. Выводы по третьей главе	82
3.4.1. Основные результаты реализации	82
Глава 4. Тестирование разработанного фреймворка автоматизации UI-тестирования	84
1. Организация и выполнение тестирования	84
4.1.1. Описание тестового окружения	84
4.1.2. Выполнение автоматизированных UI-тестов	85
2. Анализ результатов тестирования	87
4.2.1. Анализ результатов выполнения тестов	87
3. Выводы по четвертой главе	89
4.3.1. Результаты тестирования разработанного фреймворка	89
Список источников	91
Приложение А. Исходный код класса инициализации драйвера для взаимодействия с браузером, DriverFactory.java.	95
Приложение Б. Исходный код файла конфигурации фреймворка, pom.xml.	97
Приложение В. Исходный код класса для запуска тестов страницы авторизации, LoginTest.java. .	100
Приложение Г. Исходный код класса абстракции страницы авторизации, LoginPage.java.	102

Приложение Д. Ссылка на репозиторий гитхаб с исходным кодом фреймворка.	103
--	-----

Перечень используемых сокращений, обозначений, терминов

- UI (User Interface) – пользовательский интерфейс, совокупность средств взаимодействия пользователя с веб-приложением.
- UI-тестирование – вид тестирования, направленный на проверку корректности работы пользовательского интерфейса приложения.
- Selenium – программный инструмент для автоматизации управления веб-браузерами и выполнения UI-тестов.
- Selenium Grid – компонент Selenium, предназначенный для распределённого и параллельного выполнения автоматизированных тестов на нескольких browser-node.
- Selenium WebDriver – библиотека для программного взаимодействия с браузером и выполнения пользовательских действий.
- TestNG – фреймворк для организации, конфигурации и запуска автоматизированных тестов на языке Java.
- Docker – платформа контейнеризации, предназначенная для создания и запуска изолированных программных окружений.
- Docker Compose – инструмент для автоматизированного управления многоконтейнерной инфраструктурой Docker.
- Page Object Model (POM) – шаблон проектирования тестовых фреймворков, основанный на разделении логики тестов и логики взаимодействия со страницами приложения.
- Browser-session – отдельная браузерная сессия, создаваемая для выполнения тестового сценария.
- Parallel execution – параллельное выполнение тестов в нескольких потоках с целью сокращения времени тестирования.
- RemoteWebDriver – компонент Selenium WebDriver, обеспечивающий удалённое выполнение тестов через Selenium Grid.

- CI/CD (Continuous Integration / Continuous Delivery) – подход к автоматизации процессов сборки, тестирования и развертывания программного обеспечения.
- Allure Report – инструмент формирования отчётности по результатам автоматизированного тестирования.
- IDE (Integrated Development Environment) – интегрированная среда разработки программного обеспечения (в данной работе — IntelliJ IDEA).

Введение

В условиях активного развития информационных технологий и роста сложности программных систем особую значимость приобретает обеспечение качества программного обеспечения. В рамках данной работы рассматриваются вопросы тестирования именно веб-приложений и их пользовательских веб-интерфейсов. Современные веб-приложения представляют собой многоуровневые системы, включающие клиентскую часть (frontend), серверную часть (backend) и интеграцию с внешними сервисами, что существенно усложняет процесс разработки и сопровождения. Взаимодействие между frontend и backend осуществляется посредством сетевых протоколов (HTTP/HTTPS) и программных интерфейсов (API), чаще всего с использованием форматов обмена данными, таких как JSON или XML. В связи с этим автоматизация процессов тестирования становится неотъемлемой частью жизненного цикла программного обеспечения.

Тестирование программного обеспечения играет ключевую роль на всех этапах его жизненного цикла - от проектирования и разработки до внедрения и эксплуатации. Основной задачей тестирования является выявление дефектов, оценка соответствия программного продукта требованиям и снижение рисков возникновения ошибок в промышленной эксплуатации. При выполнении ручного тестирования существенную роль играет человеческий фактор: тестировщики могут допускать ошибки из-за невнимательности, усталости при выполнении повторяющихся проверок, различной интерпретации требований или пропуска граничных сценариев. Интеграция автоматизированного тестирования в процесс разработки позволяет повысить надёжность программных систем, сократить затраты на исправление ошибок и обеспечить стабильность выпускаемых версий.

Особое значение в современной разработке приобретает автоматизированное тестирование веб-приложений. Высокая частота обновлений, необходимость поддержки различных браузеров и платформ, а также тесная взаимосвязь пользовательского интерфейса с серверной

логикой требуют применения автоматизированных средств проверки. Автотестирование позволяет обеспечить воспроизводимость проверок, ускорить процесс тестирования и повысить качество программного продукта за счёт регулярного выполнения тестов при изменениях в коде.

В условиях использования практик непрерывной интеграции и доставки (CI/CD) автоматизированные тесты становятся важным элементом процесса разработки, обеспечивая оперативную обратную связь о качестве изменений. Наличие автоматизированных UI- и API-тестов позволяет своевременно выявлять дефекты, связанные с функциональностью, производительностью и корректностью работы веб-приложений.

Целью выпускной квалификационной работы является разработка и исследование фреймворка автоматизированного тестирования веб-приложений на языке Java, обеспечивающего поддержку UI-тестирования, интеграцию с CI/CD пайплайном и формирование отчётности о результатах тестирования.

Для достижения поставленной цели в работе предполагается решение следующих задач:

- анализ теоретических основ тестирования программного обеспечения и автоматизации тестирования;
- обзор и сравнительный анализ средств автоматизированного тестирования веб-приложений;
- проектирование архитектуры фреймворка автоматизированного тестирования;
- реализация основных компонентов тестовой инфраструктуры;
- интеграция разработанного фреймворка в CI/CD пайплайн;
- анализ результатов автоматизированного тестирования и формирование отчётности.

Объектом исследования в данной выпускной квалификационной работе является процесс тестирования веб-приложений.

Предметом исследования являются методы, средства и архитектурные

решения автоматизированного UI-тестирования веб-приложений с использованием языка Java.

Глава 1. Теоретические основы тестирования программного обеспечения

1. Веб-приложения и особенности их тестирования

1.1.1. Веб-интерфейсы и их роль в современных информационных системах

Веб-приложения в настоящее время являются одной из наиболее распространённых форм программных систем и широко применяются в различных сферах деятельности, включая бизнес, образование, государственные сервисы и электронную коммерцию. Веб-приложение представляет собой программный продукт, доступ к которому осуществляется через веб-браузер и который обеспечивает взаимодействие пользователя с функциональностью системы посредством сети Интернет.

Ключевым элементом веб-приложения является веб-интерфейс, представляющий собой совокупность визуальных и интерактивных элементов, обеспечивающих взаимодействие пользователя с системой. Веб-интерфейс включает в себя формы ввода данных, элементы навигации, кнопки управления и другие компоненты, через которые пользователь получает доступ к функциональности приложения. От качества реализации веб-интерфейса во многом зависит удобство использования, эффективность работы пользователя и общее восприятие программного продукта.

Архитектура современных веб-приложений, как правило, разделяется на две основные части: клиентскую и серверную. Клиентская часть, или frontend, отвечает за отображение пользовательского интерфейса, обработку пользовательских действий и первичную валидацию данных. Серверная часть, или backend, обеспечивает обработку бизнес-логики, хранение и обработку данных, а также взаимодействие с внешними сервисами и базами данных.

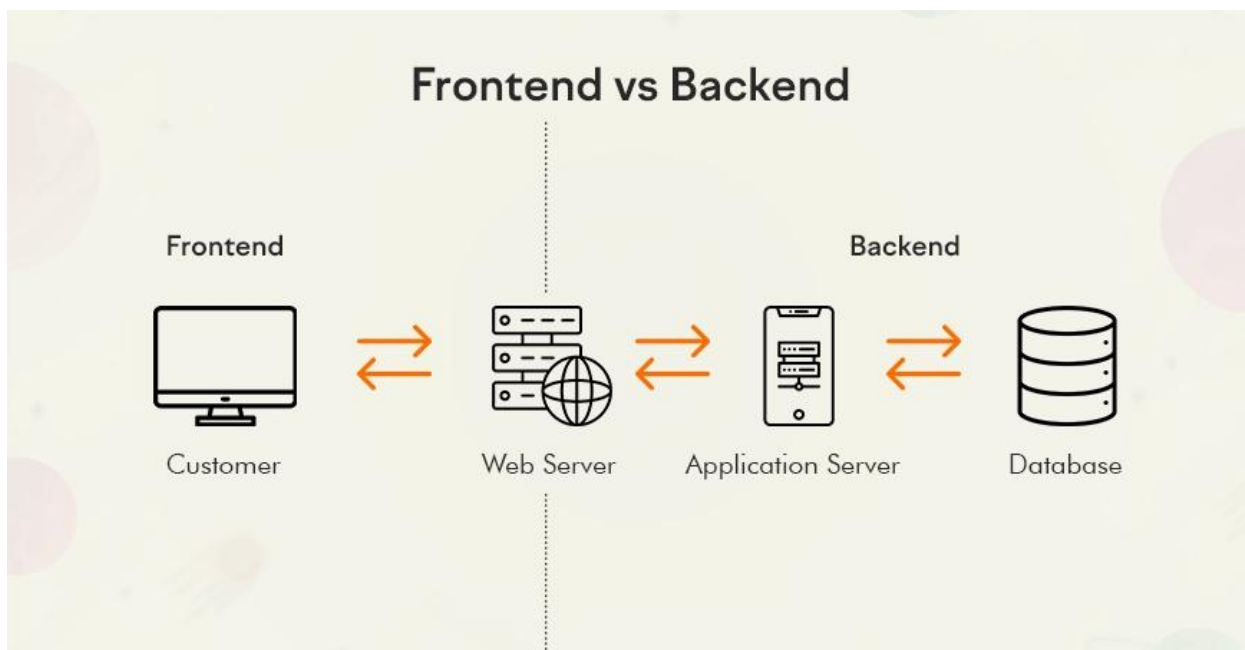


Рисунок 1.1.1. Схема взаимодействия фронтенда и бекэнда¹

Взаимодействие между клиентской и серверной частями осуществляется посредством сетевых протоколов (HTTP/HTTPS) и программных интерфейсов (API). При этом данные передаются в структурированных форматах, например JSON или XML. Нарушения в процессе обмена данными, ошибки обработки запросов или задержки ответа сервера могут приводить к некорректной работе пользовательского интерфейса, что усложняет процесс тестирования веб-приложений и требует комплексной проверки как клиентской, так и серверной части.

Веб-интерфейс является критически важной частью веб-приложения, поскольку именно через него пользователь взаимодействует с системой. Даже при корректной работе серверной логики ошибки или недостатки в пользовательском интерфейсе могут привести к снижению удобства использования, потере функциональности и негативному пользовательскому опыту. Некорректное отображение элементов, ошибки навигации или несоответствие интерфейса ожиданиям пользователя способны существенно повлиять на восприятие качества программного продукта.

¹ Frontend Vs Backend in eCommerce: What's the Difference? // BrainSpate. URL: <https://brainspate.com/blog/frontend-vs-backend-in-e-commerce-unlocking-web-development-secrets/> (дата обращения: 09.01.2026).

Качество веб-интерфейса напрямую связано с такими характеристиками, как удобство использования (UX), функциональная корректность и надёжность работы приложения. Пользовательский опыт формируется не только внешним видом интерфейса, но и стабильностью его работы, предсказуемостью поведения элементов и корректной обработкой пользовательских действий. В процессе тестирования веб-интерфейсов необходимо учитывать влияние человеческого фактора: пользователи могут вводить некорректные данные, допускать ошибки при работе с интерфейсом или использовать систему не по предполагаемому сценарию. Аналогично, при ручном тестировании тестировщики могут пропускать дефекты из-за усталости, невнимательности или повторяемости проверок. Это повышает значимость автоматизированного тестирования, позволяющего обеспечить стабильность и воспроизводимость проверок.

В связи с этим веб-интерфейс требует тщательной проверки на всех этапах разработки, что обуславливает необходимость применения систематического тестирования и, в частности, автоматизированных средств проверки.

Таким образом, веб-интерфейсы играют ключевую роль в современных информационных системах, являясь основным каналом взаимодействия пользователя с программным обеспечением. Обеспечение их качества и корректной работы является важной задачей в процессе разработки веб-приложений и напрямую влияет на эффективность и надёжность программных систем.

1.1.2. Тестирование программного обеспечения: понятие, цели и задачи

Тестирование программного обеспечения является неотъемлемой частью процесса разработки современных информационных систем. Под тестированием программного обеспечения понимается процесс проверки соответствия программного продукта заданным требованиям, выявления

дефектов и оценки качества его функционирования в различных условиях эксплуатации. Необходимость тестирования обусловлена сложностью современных программных систем и высокими требованиями к их надёжности, безопасности и удобству использования. В процессе разработки неизбежно возникают ошибки, связанные со сложностью архитектуры, изменениями требований и интеграцией различных компонентов. Основной целью тестирования является обеспечение корректной и стабильной работы программной системы до её ввода в эксплуатацию.

Основными целями тестирования программного обеспечения являются:

- выявление дефектов и ошибок на ранних этапах разработки;
- подтверждение соответствия программного продукта функциональным и нефункциональным требованиям;
- обеспечение заданного уровня качества программного обеспечения;
- снижение рисков, связанных с эксплуатацией программного продукта;
- повышение надёжности и устойчивости системы в реальных условиях использования.

Важной задачей тестирования является не только поиск ошибок, но и предотвращение их появления в дальнейшем. Раннее обнаружение дефектов позволяет значительно сократить затраты на их исправление, поскольку исправление ошибок на поздних этапах жизненного цикла программного обеспечения требует значительно больших ресурсов. Кроме того, тестирование способствует повышению качества архитектурных и проектных решений за счёт выявления слабых мест системы.

Тестирование играет ключевую роль в процессе разработки программного обеспечения и является составной частью жизненного цикла программного продукта. Оно сопровождает все этапы разработки, начиная с анализа требований и проектирования, и заканчивая внедрением и сопровождением системы. В современных подходах к разработке, таких как Agile и DevOps, тестирование интегрируется в процесс разработки и

выполняется на постоянной основе, обеспечивая быструю обратную связь и оперативное выявление дефектов.

Особую значимость тестирование приобретает при разработке веб-приложений, где корректность работы пользовательского интерфейса напрямую зависит от взаимодействия клиентской и серверной частей системы. В таких условиях тестирование должно учитывать как проверку бизнес-логики, так и корректность отображения интерфейса, обмена данными через API и поведения системы при различных сценариях использования.

Таким образом, тестирование программного обеспечения является важным инструментом обеспечения качества и надёжности программных систем. Его применение позволяет повысить устойчивость программных продуктов, снизить эксплуатационные риски и обеспечить соответствие системы ожиданиям пользователей и требованиям заказчика².

1.1.3. Виды тестирования программного обеспечения

В процессе обеспечения качества программного обеспечения применяются различные виды тестирования, направленные на проверку отдельных аспектов работы системы. Использование различных подходов позволяет выявлять дефекты на разных этапах разработки и обеспечивать стабильность программного продукта. Для веб-приложений комплексное тестирование особенно важно, поскольку такие системы включают клиентскую и серверную части, а также взаимодействуют через сеть.

² Testing // ISTQB Glossary. URL: <https://istqb-glossary.page/testing/> (дата обращения: 09.01.2026).

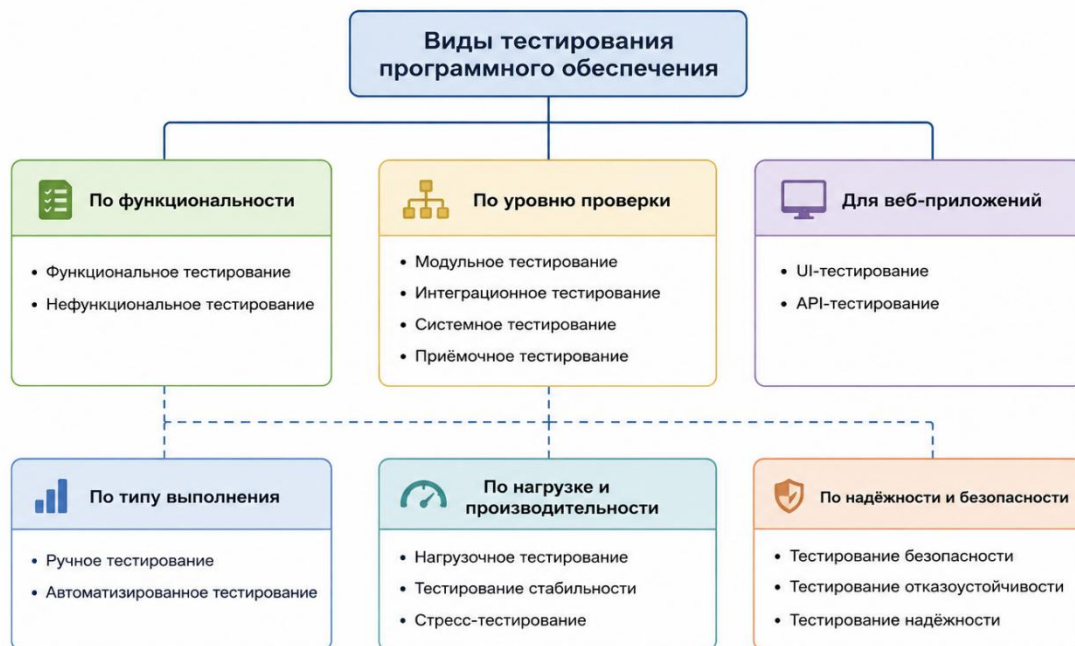


Рисунок 1.1.3. Виды тестирования

Одним из основных видов является функциональное тестирование, целью которого является проверка соответствия системы функциональным требованиям. В ходе такого тестирования оценивается корректность выполнения пользовательских сценариев, обработки данных и работы бизнес-логики.

Нефункциональное тестирование направлено на оценку характеристик системы, не связанных напрямую с функциональностью. К ним относятся производительность, безопасность, надёжность, удобство использования и устойчивость к нагрузкам. Для веб-приложений важное значение имеют нагрузочное тестирование и проверка безопасности пользовательского интерфейса.

По уровню проверки программного обеспечения выделяют модульное, интеграционное, системное и приёмочное тестирование. Модульное тестирование используется для проверки отдельных компонентов системы. Интеграционное тестирование позволяет проверить взаимодействие между модулями приложения. Системное тестирование оценивает работу

программного продукта в целом, а приёмочное тестирование подтверждает соответствие системы требованиям заказчика.

Отдельное место занимает UI-тестирование, направленное на проверку пользовательского интерфейса. В ходе такого тестирования оценивается корректность отображения элементов интерфейса, работа пользовательских сценариев и взаимодействие пользователя с системой. Для веб-приложений UI-тестирование является особенно важным, поскольку интерфейс представляет собой основной способ взаимодействия пользователя с приложением.

API-тестирование применяется для проверки программных интерфейсов приложения и корректности обмена данными между компонентами системы. Такой подход позволяет тестировать backend-часть независимо от пользовательского интерфейса и ускоряет процесс выявления дефектов.

В зависимости от способа выполнения тестирование подразделяется на ручное и автоматизированное. Ручное тестирование выполняется специалистом без использования автоматизированных средств и позволяет гибко анализировать поведение системы. Автоматизированное тестирование основано на использовании специализированных инструментов и обеспечивает высокую скорость выполнения проверок, воспроизводимость результатов и эффективность регрессионного тестирования.

Таким образом, применение различных видов тестирования обеспечивает комплексную проверку программного обеспечения и позволяет повысить качество веб-приложений.

1.1.4. Роль тестирования в разработке веб-интерфейсов

Веб-интерфейс является ключевым элементом взаимодействия пользователя с веб-приложением, поэтому его качество напрямую влияет на удобство использования, восприятие продукта и общую удовлетворённость

пользователей. Ошибки в пользовательском интерфейсе зачастую становятся наиболее заметными и критичными, даже если внутренняя логика системы функционирует корректно.

Специфика ошибок в веб-интерфейсах заключается в их разнообразии и зависимости от множества факторов. К таким ошибкам относятся некорректное отображение элементов интерфейса, неправильная работа интерактивных компонентов, нарушения адаптивной вёрстки, а также ошибки, возникающие при взаимодействии пользователя с формами, кнопками и навигационными элементами. Подобные дефекты могут приводить к невозможности выполнения пользовательских сценариев и снижению доверия к системе.

Дополнительным фактором риска является влияние человеческого фактора при тестировании пользовательских интерфейсов.³ При ручном тестировании возможно пропускание дефектов из-за усталости специалиста, невнимательности или субъективной оценки удобства использования интерфейса. Кроме того, ручная проверка большого количества пользовательских сценариев требует значительных временных затрат, что повышает вероятность неполного тестового покрытия. В связи с этим применение автоматизированных средств тестирования позволяет снизить влияние человеческого фактора и повысить стабильность результатов проверки.

Таблица 1.1.1. Факторы риска в тестировании ПО

Фактор риска	Проявление проблемы	Возможные последствия	Способы минимизации
Человеческий фактор	Пропуск дефектов, субъективная оценка UX, усталость тестировщика	Снижение качества тестирования, неполное покрытие	Использование автотестов, code review тестов, чек-листы

³ Cross Browser Testing Guide // BrowserStack. URL: <https://www.browserstack.com/guide/cross-browser-testing> (дата обращения: 09.01.2026).

Фактор риска	Проявление проблемы	Возможные последствия	Способы минимизации
Кроссбраузерность	Различия отображения UI в браузерах	Ошибки отображения, некорректная работа функций	Кроссбраузерное тестирование, Selenium Grid
Зависимость от backend	Ошибки API, задержки ответов сервера	Некорректное отображение данных, сбой UI	API-тестирование, мокирование сервисов
Частые изменения UI	Частые правки интерфейса	Ломаются тесты, растёт стоимость поддержки	Page Object Pattern, стабильные локаторы
Сложность пользовательских сценариев	Большое количество вариантов поведения	Неполное покрытие тестами	Приоритизация сценариев, риск-ориентированное тестирование

Одной из характерных проблем веб-интерфейсов является кроссбраузерность. Веб-приложения должны корректно работать в различных браузерах и на разных устройствах, однако различия в реализации стандартов HTML, CSS и JavaScript могут приводить к некорректному отображению и поведению интерфейса. В связи с этим тестирование веб-интерфейсов должно учитывать особенности популярных браузеров и их версий, а также различные разрешения экранов и типы устройств.

Важным аспектом является зависимость пользовательского интерфейса от серверной части приложения. Веб-интерфейс тесно связан с backend-компонентами, поскольку данные для отображения, результаты пользовательских действий и состояние интерфейса формируются на основе ответов сервера. Взаимодействие между frontend и backend, как правило, осуществляется через API с использованием сетевых протоколов и форматов обмена данными. Ошибки в API, задержки обработки запросов или

некорректная обработка данных на сервере могут напрямую отражаться на работе интерфейса, что требует комплексного подхода к тестированию и проверки взаимодействия frontend и backend частей системы.

С учётом сложности современных веб-приложений и высокой частоты изменений в пользовательском интерфейсе возрастает необходимость автоматизации UI-тестирования. Автоматизированные UI-тесты позволяют регулярно проверять основные пользовательские сценарии, обеспечивать стабильность функциональности при внесении изменений и сокращать затраты времени на регрессионное тестирование. Использование средств автоматизации тестирования способствует повышению качества веб-интерфейсов и снижению рисков появления критических дефектов в процессе эксплуатации приложения.

Таким образом, тестирование веб-интерфейсов является неотъемлемой частью разработки веб-приложений и требует системного подхода, учитывающего специфику ошибок, проблемы кроссбраузерности, взаимосвязь с серверной логикой, влияние человеческого фактора и необходимость применения автоматизированных методов тестирования.

2. Автоматизированное тестирование программного обеспечения

1.2.1. Понятие и подходы к автоматизированному тестированию

Автоматизированное тестирование программного обеспечения представляет собой процесс выполнения тестовых сценариев с использованием специализированных программных средств без непосредственного участия человека. В отличие от ручного тестирования, при котором тестировщик самостоятельно выполняет действия и анализирует результаты, автоматизированное тестирование позволяет запускать тесты автоматически, обеспечивая воспроизводимость и масштабируемость процесса проверки качества программного продукта.

Автоматизация тестирования целесообразна в случаях, когда требуется регулярное выполнение однотипных тестовых сценариев, например при регрессионном тестировании, проверке критически важных бизнес-функций и многократных сборках программного обеспечения.⁴ Особенно эффективно автоматизированное тестирование применяется в проектах с длительным жизненным циклом, высокой частотой изменений и необходимостью быстрого получения обратной связи о качестве продукта.

К основным преимуществам автоматизированного тестирования относятся:

- сокращение времени выполнения тестов по сравнению с ручным тестированием;
- повышение стабильности и повторяемости тестовых сценариев;
- снижение влияния человеческого фактора;
- возможность интеграции тестирования в процессы непрерывной интеграции и доставки (CI/CD);
- ускорение выявления дефектов на ранних этапах разработки.

Однако автоматизация тестирования имеет и ряд ограничений. К недостаткам относятся значительные начальные затраты на разработку и поддержку автотестов, необходимость наличия квалифицированных специалистов, а также ограниченная применимость автоматизации для тестирования пользовательского опыта и визуального восприятия интерфейса. В связи с этим автоматизированное тестирование не заменяет полностью ручное тестирование, а дополняет его.

Существуют различные подходы к автоматизированному тестированию, выбор которых зависит от архитектуры системы и целей тестирования. Наиболее распространёнными являются:

- автоматизация тестирования на уровне пользовательского интерфейса;
- автоматизация тестирования API и сервисного уровня;

⁴ Continuous Integration // Atlassian. URL: <https://www.atlassian.com/continuous-delivery/continuous-integration> (дата обращения: 09.01.2026).

- автоматизация модульного и интеграционного тестирования.

В современных методологиях разработки программного обеспечения автоматизированное тестирование является неотъемлемой частью жизненного цикла программного обеспечения (SDLC). Автотесты разрабатываются и поддерживаются параллельно с основным кодом приложения и используются на различных этапах разработки — от локального тестирования до промышленной эксплуатации.

Особую роль автоматизированное тестирование играет в процессах непрерывной интеграции и непрерывной доставки (CI/CD). В рамках CI/CD автотесты автоматически запускаются при каждом изменении кода, что позволяет оперативно выявлять дефекты, предотвращать попадание некорректных изменений в основную ветку разработки и обеспечивать стабильность выпускаемых версий программного обеспечения.

Таким образом, автоматизированное тестирование является эффективным инструментом повышения качества программного обеспечения, позволяющим оптимизировать процессы тестирования и разработки при условии его грамотного применения и сочетания с ручными методами тестирования.

1.2.2. Виды автоматизированного тестирования

Автоматизированное тестирование включает в себя различные виды тестов, которые применяются на разных уровнях программной системы и направлены на проверку отдельных компонентов, их взаимодействия и поведения системы в целом. Разделение автоматизированных тестов по уровням позволяет выстроить эффективную стратегию обеспечения качества программного обеспечения.⁵

⁵ ISTQB® Certified Tester Foundation Level Syllabus Version 4.0 // ISTQB. URL: <https://istqb.org/certifications/certified-tester-foundation-level/> (дата обращения: 09.01.2026).



Рисунок 1.2.1. Пирамида тестирования

Модульные автотесты предназначены для проверки отдельных компонентов или методов программы в изоляции от остальных частей системы. Такие тесты позволяют выявлять ошибки на ранних этапах разработки, обеспечивая корректность работы базовой логики приложения. Модульные автотесты отличаются высокой скоростью выполнения и простотой поддержки, что делает их важной частью процесса разработки.

Интеграционные автотесты направлены на проверку корректности взаимодействия между несколькими модулями или компонентами системы. В рамках интеграционного тестирования проверяется обмен данными между слоями приложения, взаимодействие с базами данных и внешними сервисами. Эти тесты позволяют выявлять ошибки, которые невозможно обнаружить на уровне отдельных модулей.

UI-автотесты ориентированы на проверку пользовательского интерфейса веб-приложения. Они имитируют действия реального

пользователя, такие как навигация по страницам, ввод данных в формы и выполнение пользовательских сценариев. UI-автотесты позволяют убедиться в корректной работе интерфейса, однако требуют значительных ресурсов на разработку и поддержку, поскольку чувствительны к изменениям в структуре и дизайне интерфейса.

API-автотесты используются для проверки серверной части приложения и взаимодействия через программные интерфейсы. Такие тесты позволяют проверить корректность обработки запросов, структуру ответов, работу механизмов авторизации и обработку ошибок. API-тестирование является важным этапом обеспечения стабильности backend-компонентов и часто выполняется независимо от пользовательского интерфейса.

End-to-end тестирование направлено на проверку полной цепочки работы системы от пользовательского интерфейса до серверной части и базы данных. End-to-end автотесты моделируют реальные пользовательские сценарии и позволяют убедиться, что все компоненты системы корректно взаимодействуют между собой. Несмотря на высокую ценность таких тестов, они обладают высокой стоимостью разработки и исполнения, поэтому применяются выборочно для проверки ключевых бизнес-сценариев.

Таким образом, использование различных видов автоматизированного тестирования позволяет обеспечить комплексную проверку программного продукта на всех уровнях его архитектуры и повысить общее качество и надёжность веб-приложений.

1.2.3. Обзор средств автоматизированного тестирования

Для реализации автоматизированного тестирования веб-приложений используется широкий набор инструментальных средств, которые позволяют автоматизировать проверку пользовательского интерфейса, серверной логики,

а также организовать процесс непрерывного тестирования и анализа результатов.⁶

Одним из наиболее распространённых инструментов для автоматизации тестирования веб-интерфейсов является Selenium. Selenium представляет собой набор программных компонентов и библиотек, предназначенных для автоматизации взаимодействия с веб-браузерами. Данный инструмент позволяет программно управлять действиями пользователя: открывать веб-страницы, выполнять ввод данных, нажимать кнопки, переходить между страницами и проверять состояние элементов интерфейса.

Основным компонентом Selenium является Selenium WebDriver, обеспечивающий взаимодействие тестового кода с браузером через специальные драйверы. WebDriver поддерживает работу с различными браузерами, включая Google Chrome, Mozilla Firefox, Microsoft Edge и другие. Благодаря этому Selenium широко используется для кроссбраузерного тестирования веб-приложений.

Дополнительно Selenium включает компонент Selenium Grid, предназначенный для распределённого и параллельного выполнения тестов. Selenium Grid позволяет запускать тестовые сценарии одновременно на нескольких браузерах и узлах инфраструктуры, что существенно сокращает время выполнения регрессионного тестирования. Использование Selenium совместно с Docker позволяет организовать масштабируемую контейнеризированную среду тестирования.

Одним из преимуществ Selenium является поддержка различных языков программирования, включая Java, Python, C# и JavaScript. В рамках данной работы Selenium используется совместно с языком Java и фреймворком TestNG для построения архитектуры автоматизированного UI-тестирования.

⁶ Selenium Documentation // Selenium. URL: <https://www.selenium.dev/documentation/> (дата обращения: 09.01.2026).

Для организации и управления автотестами используются фреймворки модульного тестирования, такие как TestNG и JUnit.⁷ Эти инструменты позволяют структурировать тесты, управлять порядком их выполнения, использовать аннотации и формировать отчётность о результатах тестирования. В контексте автоматизированного тестирования веб-приложений данные фреймворки применяются в качестве основы для построения тестовой инфраструктуры.

Для тестирования программных интерфейсов приложений применяются специализированные инструменты API-тестирования. Они позволяют формировать HTTP-запросы, проверять корректность ответов сервера, а также выполнять валидацию данных и механизмов авторизации. Использование API-тестирования позволяет проверить бизнес-логику приложения независимо от пользовательского интерфейса и ускорить процесс выявления дефектов.

Важной составляющей автоматизированного тестирования являются инструменты непрерывной интеграции и доставки (CI/CD). Одним из наиболее распространённых решений в данной области является Jenkins, который позволяет автоматизировать процесс сборки проекта и запуск автотестов при изменениях в коде. Интеграция автотестов с CI/CD-системами обеспечивает регулярную проверку качества программного обеспечения и своевременное выявление ошибок.⁸

Для анализа результатов тестирования и наглядного представления информации о качестве программного продукта используются инструменты отчётности, такие как Allure Report. Данные средства позволяют формировать подробные отчёты о выполнении тестов, отображать статистику, историю запусков и выявленные дефекты. Наличие отчётности облегчает анализ результатов тестирования и повышает прозрачность процесса обеспечения качества.

⁷ JUnit 5 User Guide // JUnit. URL: <https://junit.org/junit5/docs/current/user-guide/> (дата обращения: 09.01.2026).

⁸ Jenkins Documentation // Jenkins. URL: <https://www.jenkins.io/doc/> (дата обращения: 09.01.2026).

Таблица 1.2.1. Средства автоматизированного тестирования

Инструмент	Тип тестирования	Основное назначение	Преимущества	Область применения
Selenium	UI-тестирование	Автоматизация действий пользователя в браузере	Кроссбраузерность, поддержка многих языков	Тестирование веб-интерфейсов
JUnit	Модульное тестирование	Организация и запуск тестов	Простота интеграции, распространённость	Backend-логика, unit-тесты
TestNG	Модульное тестирование	Управление тестовыми наборами	Гибкая конфигурация, отчётность	Комплексные тестовые сценарии
API-тест инструменты	API-тестирование	Проверка HTTP-запросов и ответов	Быстрое выявление ошибок бизнес-логики	Проверка backend без UI
Jenkins	CI/CD	Автоматизация сборки и запуска тестов	Автоматизация процессов разработки	Непрерывная интеграция
Allure Report	Отчётность	Анализ результатов тестирования	Наглядные отчёты, история запусков	Анализ качества ПО

Таким образом, использование современных средств автоматизированного тестирования позволяет выстроить эффективную и масштабируемую систему контроля качества веб-приложений, интегрированную в процесс разработки и сопровождения программного обеспечения.

1.2.4. Преимущества и ограничения автоматизированного тестирования

Автоматизированное тестирование обладает рядом существенных преимуществ, которые делают его эффективным инструментом обеспечения

качества программного обеспечения, особенно в проектах с высокой сложностью и частыми изменениями.

К основным преимуществам автоматизированного тестирования относится его высокая эффективность при повторяющихся проверках. Автотесты позволяют выполнять регрессионное тестирование после каждого изменения кода без значительных временных затрат. Это особенно важно для крупных веб-приложений, где ручная проверка всех функциональных сценариев требует значительных ресурсов.⁹

Автоматизация тестирования также обеспечивает стабильность и воспроизводимость результатов. Автотесты выполняются по заранее заданным сценариям и исключают влияние человеческого фактора, что позволяет получать объективную информацию о состоянии системы. Кроме того, автоматизированное тестирование хорошо интегрируется в процессы непрерывной интеграции и доставки, обеспечивая быстрый контроль качества на каждом этапе разработки.

Наиболее эффективно автоматизированное тестирование применяется для проверки бизнес-критичных сценариев, API-интерфейсов, а также при нагрузочном и регрессионном тестировании. В таких случаях автоматизация позволяет существенно сократить время тестирования и повысить надёжность выпускаемых версий программного обеспечения.

В то же время автоматизированное тестирование имеет и определённые ограничения. Автотесты не способны полностью заменить ручное тестирование, особенно в задачах, связанных с оценкой удобства использования, визуального восприятия интерфейса и пользовательского опыта. Такие аспекты требуют участия человека и субъективной оценки.

Существенной проблемой автоматизированного тестирования является сложность поддержки автотестов. Изменения в пользовательском интерфейсе, бизнес-логике или архитектуре приложения могут приводить к

⁹ Benefits and Limitations of Test Automation // BrowserStack. URL: <https://www.browserstack.com/guide/test-automation> (дата обращения: 09.01.2026).

необходимости доработки большого количества тестов. Без продуманной архитектуры тестовой инфраструктуры автотесты могут становиться нестабильными и трудоёмкими в сопровождении.

В связи с этим эффективное применение автоматизированного тестирования требует архитектурного подхода к построению тестовой системы. Необходимо учитывать принципы модульности, переиспользуемости компонентов и изоляции тестов. Грамотно спроектированная тестовая инфраструктура позволяет снизить затраты на поддержку автотестов и обеспечить их долгосрочную эффективность.

Таким образом, автоматизированное тестирование является мощным инструментом обеспечения качества программного обеспечения, однако его использование должно быть обоснованным и сочетаться с ручными методами тестирования в рамках комплексной стратегии контроля качества.

3. Выбор средств и технологий для разработки автотестов

1.3.1. Критерии выбора инструментов автоматизации тестирования

Выбор инструментов автоматизации тестирования является важным этапом при построении тестовой инфраструктуры. От используемых технологий зависят удобство сопровождения автотестов, возможность масштабирования системы и интеграция с процессами разработки.

Одним из ключевых критериев является язык программирования. Используемый язык должен быть распространённым, обладать развитой экосистемой и поддерживаться выбранными средствами автоматизации. Применение единого языка для разработки приложения и автотестов упрощает сопровождение проекта.

Важным фактором также является масштабируемость тестовой инфраструктуры. Инструменты должны обеспечивать возможность

расширения набора тестов и поддержки различных окружений без существенного изменения архитектуры.

Современные средства автоматизации должны поддерживать интеграцию с CI/CD-системами, позволяя автоматически запускать тесты в процессе сборки и получать быструю обратную связь о качестве программного продукта.

Дополнительное значение имеет поддержка отчётности. Формирование наглядных отчётов упрощает анализ результатов тестирования, контроль ошибок и оценку стабильности системы.

Также важным критерием является возможность параллельного выполнения тестов, позволяющая сократить общее время тестирования и повысить эффективность работы тестовой инфраструктуры.

Таблица 1.3.1. Критерии выбора инструментов автоматизации тестирования

Критерий	Содержание критерия	Практическое значение
Язык программирования	Соответствие используемого инструмента языку разработки проекта	Упрощение поддержки тестового кода, снижение порога входа
Масштабируемость	Возможность расширения тестовой инфраструктуры и добавления новых тестов	Обеспечение роста системы тестирования вместе с проектом
Интеграция с CI/CD	Возможность автоматического запуска тестов в процессе сборки	Быстрое выявление ошибок и повышение качества релизов
Поддержка отчётности	Формирование наглядных отчётов о результатах тестирования	Упрощение анализа дефектов и контроля качества
Параллельный запуск тестов	Возможность одновременного выполнения нескольких тестов	Сокращение времени тестирования

Таким образом, при выборе инструментов автоматизации тестирования необходимо учитывать совокупность критериев, обеспечивающих

эффективность, надёжность и долгосрочную поддерживаемость тестовой инфраструктуры.

1.3.2. Обоснование выбора стека для разработки автотестов

При разработке системы автоматизированного тестирования веб-приложений особое значение имеет выбор технологического стека, который должен обеспечивать надёжность, расширяемость и удобство интеграции в процессы разработки. Выбор инструментов осуществляется с учётом критериев, рассмотренных в предыдущем разделе, а также их распространённости и применимости в промышленной разработке.

В качестве основного языка программирования для разработки автотестов был выбран Java. Данный язык широко используется в корпоративных и веб-проектах, обладает высокой стабильностью, развитой экосистемой библиотек и инструментов, а также поддерживается большинством популярных фреймворков автоматизированного тестирования. Использование Java позволяет разрабатывать масштабируемые и поддерживаемые автотесты, а также интегрировать их с существующими системами сборки и CI/CD.

Для автоматизации тестирования пользовательского интерфейса веб-приложений используется Selenium. Этот инструмент предоставляет возможность управления веб-браузерами и имитации действий пользователя, что позволяет реализовывать UI-автотесты, близкие к реальным пользовательским сценариям. Selenium поддерживает различные браузеры и операционные системы, что делает его удобным средством для кроссбраузерного тестирования.

В качестве фреймворка управления тестами выбран TestNG. Он предоставляет средства для структурирования тестовых сценариев, управления порядком их выполнения, группировки тестов и реализации

параметризации. TestNG также поддерживает параллельный запуск тестов, что позволяет сократить общее время выполнения тестового набора.

Для обеспечения воспроизводимости окружения и упрощения развертывания тестовой инфраструктуры применяется Docker. Контейнеризация позволяет изолировать тестовое окружение, обеспечить единые условия выполнения тестов и упростить интеграцию автотестов в процессы непрерывной интеграции.

Для реализации параллельного и распределённого запуска UI-автотестов используется Selenium Grid. Данный инструмент позволяет выполнять тесты одновременно в нескольких браузерах и на различных конфигурациях, что значительно ускоряет процесс тестирования и повышает покрытие сценариев.

Интеграция автотестов в процесс непрерывной интеграции осуществляется с использованием системы Jenkins. Jenkins позволяет автоматизировать запуск тестов при изменениях в коде, управлять тестовыми заданиями и получать оперативную информацию о состоянии тестирования в рамках CI/CD-процесса.

Для анализа результатов тестирования и представления информации в наглядном виде используется Allure Report. Данный инструмент формирует подробные отчёты о выполнении автотестов, включая информацию о статусе тестов, шагах выполнения и выявленных ошибках, что облегчает анализ качества программного обеспечения.

Таким образом, выбранный стек технологий обеспечивает комплексный подход к автоматизированному тестированию веб-приложений и соответствует современным требованиям к качеству, масштабируемости и интеграции тестовой инфраструктуры.

4. Выводы по первой главе

В первой главе выпускной квалификационной работы были рассмотрены теоретические основы тестирования программного обеспечения с акцентом на особенности тестирования веб-приложений и применение

автоматизированных методов обеспечения качества. Проведённый анализ позволил определить роль тестирования как неотъемлемой части жизненного цикла программного обеспечения и обосновать его значимость в процессе разработки современных информационных систем.

В рамках главы были рассмотрены основные виды тестирования программного обеспечения, включая функциональное и нефункциональное тестирование, а также модульное, интеграционное, системное и приёмочное тестирование. Отдельное внимание было уделено UI- и API-тестированию, которые играют ключевую роль в обеспечении качества веб-приложений. Рассмотрены особенности ошибок, характерных для веб-интерфейсов, проблемы кроссбраузерности и зависимость пользовательского интерфейса от серверной части приложения.

Также были изучены принципы и подходы к автоматизированному тестированию, его преимущества и ограничения. Показано, что автоматизация тестирования является эффективным инструментом для проверки повторяющихся сценариев и регрессионного тестирования, однако требует архитектурного подхода и не может полностью заменить ручное тестирование. Обоснована необходимость сочетания различных видов тестирования в рамках комплексной стратегии обеспечения качества программного обеспечения.

В теоретической части был выполнен обзор современных средств автоматизированного тестирования и определены критерии их выбора. На основе проведённого анализа обоснован выбор технологического стека для разработки системы автотестов, включающего язык программирования Java, инструменты Selenium и TestNG, а также средства контейнеризации, распределённого запуска тестов и CI/CD-интеграции.

Таким образом, теоретическая часть работы сформировала основу для дальнейшего проектирования системы автоматизированного тестирования веб-приложений. Полученные выводы и обоснования послужили базой для

разработки архитектуры тестовой инфраструктуры, которая будет подробно рассмотрена во второй главе выпускной квалификационной работы.

Глава 2. Теоретические основы тестирования программного обеспечения

1. Анализ требований к системе автоматизированного тестирования

2.1.1. Цели и задачи разрабатываемого фреймворка

В современных условиях разработки веб-приложения регулярно обновляются и изменяются, что требует постоянного проведения регрессионного тестирования. Выполнение подобных проверок вручную занимает значительное количество времени и снижает эффективность процесса тестирования. В связи с этим возрастает необходимость использования автоматизированных средств проверки пользовательского интерфейса.

Целью данной работы является разработка фреймворка автоматизации UI-тестирования веб-приложений на языке Java с использованием Selenium WebDriver, TestNG, Selenium Grid и Docker.

Разрабатываемый фреймворк предназначен для автоматизированного выполнения UI-тестов, повышения стабильности тестирования и сокращения времени проверки веб-приложения.

Для достижения поставленной цели были определены следующие задачи:

- разработать архитектуру фреймворка автоматизированного тестирования;
- обеспечить поддержку автоматизированного выполнения UI-тестов веб-приложения;
- реализовать механизм параллельного выполнения тестов;
- обеспечить возможность распределённого запуска тестов с использованием Selenium Grid;
- реализовать контейнеризацию тестовой инфраструктуры с использованием Docker;
- обеспечить централизованное управление запуском тестов;

- реализовать механизм формирования отчётов о результатах тестирования;
- обеспечить масштабируемость и расширяемость тестового фреймворка;
- повысить стабильность выполнения UI-тестов за счёт использования архитектурных шаблонов и централизованного управления WebDriver.

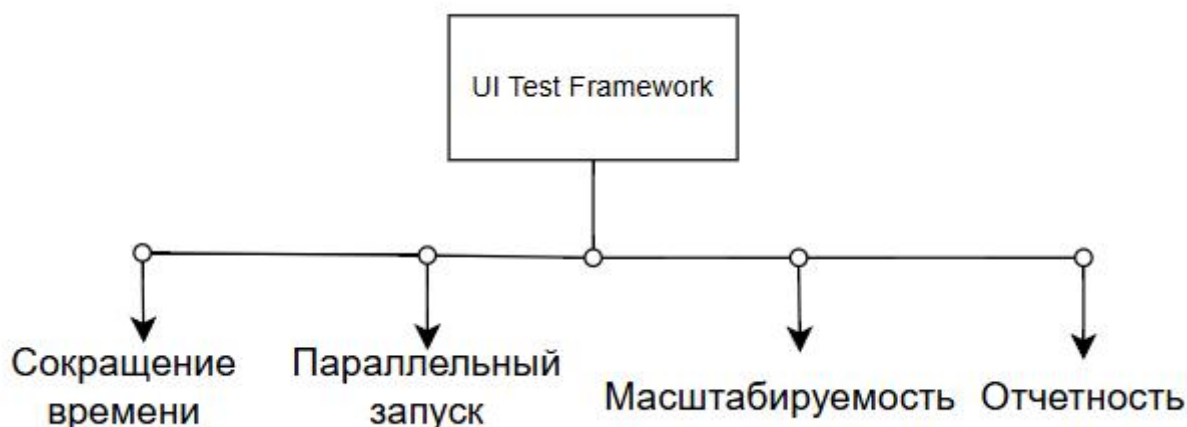


Рисунок 2.1.1. Основные цели фреймворка автоматизации тестирования

Одной из ключевых задач является сокращение времени регрессионного тестирования за счёт параллельного и распределённого запуска тестов. Дополнительное внимание уделяется повышению стабильности UI-проверок посредством использования архитектурного подхода Page Object Model и централизованного управления WebDriver.

Также важным требованием является возможность масштабирования тестовой инфраструктуры и подключения дополнительных browser-node без изменения архитектуры системы. Для анализа результатов тестирования во фреймворке реализуется автоматическая генерация отчётов с использованием Allure Reports.

Таким образом, разрабатываемый фреймворк должен обеспечить повышение эффективности UI-тестирования, снижение трудозатрат на выполнение регрессионных проверок и создание масштабируемой инфраструктуры автоматизированного тестирования.

2.1.2. Требования к архитектуре тестового фреймворка

При разработке фреймворка автоматизации UI-тестирования для веб-приложения SauceDemo особое внимание уделялось архитектуре системы. От выбранного архитектурного подхода зависят стабильность тестов, удобство сопровождения проекта и возможность масштабирования тестовой инфраструктуры.

Таблица 2.1.2. Таблица требований к архитектуре

Требование	Описание
Модульность	Разделение компонентов
Расширяемость	Добавление новых тестов
Поддерживаемость	Удобство сопровождения

В процессе проектирования были сформированы основные требования к архитектуре разрабатываемого тестового фреймворка.

Модульность

Одним из ключевых требований является модульность архитектуры. Фреймворк должен состоять из отдельных логически независимых компонентов, отвечающих за выполнение конкретных задач. Разделение системы на модули позволяет упростить сопровождение проекта, повысить читаемость кода и снизить уровень связанности между компонентами.

В рамках разрабатываемого решения отдельные модули отвечают за:

- управление браузерами и WebDriver;
- хранение тестовых данных;
- реализацию Page Object классов;
- выполнение тестовых сценариев;
- генерацию отчётности;
- обработку конфигурационных параметров;
- интеграцию с Selenium Grid.

Подобный подход позволяет изменять или расширять отдельные части системы без необходимости переработки всего фреймворка.

Одним из основных требований является модульность архитектуры. Фреймворк разделён на отдельные компоненты, отвечающие за управление драйверами, Page Object классы, выполнение тестов, хранение конфигурации и формирование отчётности. Такой подход упрощает сопровождение и развитие проекта.

Важным требованием также является расширяемость. Архитектура должна обеспечивать возможность добавления новых тестовых сценариев, страниц и браузеров без существенного изменения существующего кода. Для этого используется шаблон Page Object Model, базовые классы и централизованное управление WebDriver.

Для уменьшения дублирования кода архитектура поддерживает повторное использование компонентов. Общие действия и методы вынесены в базовые классы и utility-компоненты, что повышает стабильность тестов и упрощает разработку.

Поддерживаемость системы обеспечивается за счёт разделения тестовой логики и элементов интерфейса. При изменении пользовательского интерфейса обновление локаторов выполняется только в соответствующих Page Object классах без изменения тестовых сценариев.

Дополнительным требованием является независимость тестов. Каждый тест выполняется в отдельной browser-session и не зависит от результатов других сценариев, что особенно важно при параллельном запуске тестов через Selenium Grid.

Архитектура фреймворка также должна обеспечивать удобную интеграцию с CI/CD-системами. Использование Docker и Selenium Grid позволяет запускать тесты в изолированной и воспроизводимой среде, а также упрощает автоматизацию тестирования в Jenkins и других системах непрерывной интеграции.

Таким образом, сформированные требования направлены на создание масштабируемого, поддерживаемого и стабильного фреймворка автоматизации UI-тестирования.

2.1.3. Требования к архитектуре тестового фреймворка

При проектировании фреймворка автоматизации UI-тестирования были определены функциональные и нефункциональные требования, определяющие возможности системы, особенности её работы и требования к качественным характеристикам программного решения.

Таблица 2.1.3. Таблица функциональных и нефункциональных требований

Требование	Описание
Функциональное	Выполнение UI-тестов
Функциональное	Формирование отчетов
Нефункциональное	Масштабируемость

Сформированные требования учитывают специфику автоматизированного тестирования веб-приложений и особенности распределённого выполнения тестов с использованием Selenium Grid и Docker.

Функциональные требования определяют базовые возможности системы автоматизированного тестирования.

Фреймворк должен обеспечивать запуск UI-тестов с использованием TestNG, включая выполнение отдельных сценариев и полных наборов регрессионных проверок. Также система должна поддерживать работу с веб-браузерами через Selenium WebDriver, включая локальный и удалённый запуск через Selenium Grid с использованием RemoteWebDriver.

В рамках тестирования веб-приложения SauceDemo должны выполняться сценарии, охватывающие авторизацию пользователей, работу с каталогом товаров, сортировку, корзину и оформление заказа. При этом система должна поддерживать как позитивные, так и негативные сценарии.

Важным требованием является формирование отчётности о результатах выполнения тестов. Система должна фиксировать статус тестов, ошибки,

время выполнения и шаги тестирования. Для этого используется Allure Reports.

Также фреймворк должен поддерживать параллельное выполнение тестов с использованием возможностей TestNG, обеспечивая многопоточность и корректную работу WebDriver в конкурентной среде.

Нефункциональные требования определяют качество и эксплуатационные характеристики системы.

Фреймворк должен обеспечивать высокую производительность за счёт параллельного и распределённого выполнения тестов с использованием Selenium Grid и Docker, что позволяет сократить время регрессионного тестирования.

Система должна быть стабильной и минимизировать количество нестабильных падений тестов за счёт использования ожиданий, изоляции тестов и централизованного управления WebDriver.

Архитектура должна быть масштабируемой и позволять добавлять новые browser-node без изменения существующих тестов. Также важна переносимость системы, обеспечиваемая использованием Docker, что позволяет запускать тесты в различных окружениях без дополнительной настройки.

Дополнительно фреймворк должен быть удобным в сопровождении за счёт использования Page Object Model, разделения логики компонентов и централизованной конфигурации.

Таким образом, требования формируют основу для построения устойчивого, масштабируемого и поддерживаемого фреймворка автоматизированного тестирования веб-приложений.

2. Проектирование архитектуры фреймворка автоматизации тестирования

2.2.1. Выбор архитектурного подхода к построению фреймворка

При разработке фреймворка автоматизации UI-тестирования особое внимание уделялось выбору архитектурного подхода к организации тестового проекта. Архитектура фреймворка должна обеспечивать удобство сопровождения, расширяемость, повторное использование компонентов и стабильность выполнения тестов.

В качестве основного архитектурного подхода при разработке системы был выбран шаблон проектирования Page Object Model (POM). Данный подход является одним из наиболее распространённых решений при разработке UI-автотестов с использованием Selenium WebDriver и применяется для структурирования взаимодействия тестов с пользовательским интерфейсом веб-приложения.

Суть подхода Page Object Model заключается в представлении каждой страницы веб-приложения в виде отдельного класса, содержащего:

- локаторы элементов интерфейса;
- методы взаимодействия с элементами страницы;
- бизнес-операции, выполняемые пользователем.

В рамках разрабатываемого фреймворка для веб-приложения SauceDemo отдельные Page Object классы реализованы для страниц:

- авторизации;
- каталога товаров;
- корзины;
- оформления заказа;
- подтверждения заказа.

Причины выбора Page Object Model

Выбор Page Object Model обусловлен рядом преимуществ данного подхода при разработке автоматизированных UI-тестов.

Во-первых, использование POM обеспечивает разделение тестовой логики и логики взаимодействия с пользовательским интерфейсом. Тестовые

сценарии содержат только последовательность действий и проверок, тогда как работа с элементами страницы инкапсулируется внутри Page Object классов.

Например, тест авторизации описывает только бизнес-сценарий:

- ввод логина;
- ввод пароля;
- нажатие кнопки входа;
- проверка успешной авторизации.

При этом локаторы полей ввода и кнопок хранятся внутри соответствующего Page Object класса. Такой подход повышает читаемость тестов и делает их более понятными.

Во-вторых, применение POM позволяет существенно уменьшить дублирование кода. Одни и те же элементы интерфейса и действия пользователя могут использоваться в различных тестовых сценариях. Без применения архитектурного шаблона локаторы и методы взаимодействия пришлось бы повторять во множестве тестов.

Использование Page Object классов позволяет централизовать работу с элементами интерфейса и повторно использовать уже реализованные методы.

Преимущества использования POM

Применение Page Object Model в разрабатываемом фреймворке обеспечивает следующие преимущества:

- упрощение сопровождения тестового проекта;
- снижение связанности между тестами и пользовательским интерфейсом;
- уменьшение объёма дублируемого кода;
- повышение читаемости тестовых сценариев;
- упрощение масштабирования тестового набора;
- централизованное управление локаторами элементов;
- повышение стабильности тестов при изменении интерфейса приложения.

Особенно важным преимуществом является упрощение сопровождения тестов при изменении структуры веб-интерфейса. В случае изменения

локаторов элементов достаточно обновить соответствующий Page Object класс без необходимости изменения тестовых сценариев.

Использование базовых компонентов фреймворка

Для повышения уровня переиспользования компонентов и упрощения структуры проекта в архитектуре фреймворка используются базовые классы и вспомогательные компоненты.

BasePage

Класс BasePage используется в качестве базового класса для всех Page Object компонентов. В данном классе реализуются общие методы взаимодействия с элементами пользовательского интерфейса:

- поиск элементов;
- ожидание появления элементов;
- нажатие кнопок;
- ввод текста;
- получение текстовых значений;
- проверка состояния элементов.

Использование BasePage позволяет централизовать общую функциональность и избежать повторения одинаковых методов в различных Page Object классах.

BaseTest

Класс BaseTest используется в качестве базового класса для тестовых сценариев. В нём реализуется:

- инициализация WebDriver;
- настройка браузера;
- подключение к Selenium Grid;
- управление жизненным циклом тестов;
- запуск и завершение browser-session.

Дополнительно BaseTest обеспечивает подготовку тестового окружения перед выполнением тестов и освобождение ресурсов после завершения тестирования.

Использование utility-классов

Для реализации вспомогательной функциональности во фреймворке используются utility-классы. Подобные компоненты предназначены для выполнения общих операций, не связанных напрямую с конкретными страницами приложения.

Utility-классы могут использоваться для:

- чтения конфигурационных параметров;
- работы с тестовыми данными;
- генерации случайных значений;
- выполнения дополнительных проверок;
- логирования;
- работы со скриншотами;
- взаимодействия с системой отчётности Allure.

Использование вспомогательных компонентов позволяет повысить модульность проекта и упростить сопровождение системы автоматизированного тестирования.

Таким образом, применение архитектурного подхода Page Object Model совместно с использованием базовых классов и вспомогательных компонентов позволяет создать масштабируемый, поддерживаемый и устойчивый фреймворк автоматизации UI-тестирования веб-приложений.

2.2.2. Общая архитектура системы автоматизированного тестирования

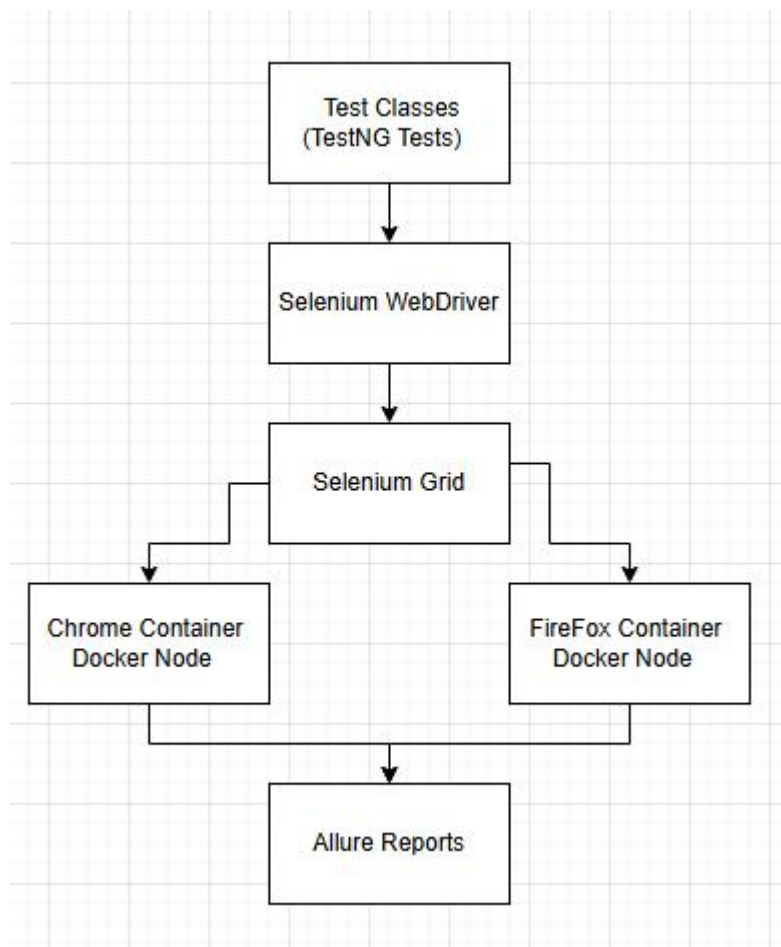


Рисунок 2.2.2.1 Архитектура фреймворка

Архитектура разработанного фреймворка автоматизированного тестирования включает набор взаимосвязанных компонентов, обеспечивающих выполнение UI-тестов, распределённый запуск браузеров и формирование отчётности. Общая схема взаимодействия компонентов системы представлена на рисунке 2.2.2.1 Также на рисунке 2.2.2.2. представлена диаграмма прецедентов.



Рисунок 2.2.2.2 Диаграмма прецедентов

В основе системы лежит фреймворк TestNG, отвечающий за управление выполнением тестовых сценариев. TestNG обеспечивает запуск тестов, управление жизненным циклом тестирования, поддержку параллельного выполнения и формирование тестовых наборов. С помощью файла testng.xml задаются параметры запуска тестов, количество потоков и последовательность выполнения тестовых классов.

Для автоматизации взаимодействия с веб-интерфейсом используется Selenium WebDriver. Данный компонент обеспечивает управление браузером и выполнение пользовательских действий:

- открытие страниц;
- ввод данных;
- нажатие элементов интерфейса;
- выполнение проверок.

Тестовые сценарии используют Selenium WebDriver для отправки команд браузеру и получения результатов выполнения операций.

Для организации распределённого выполнения тестов используется Selenium Grid. Selenium Grid выступает в роли промежуточного компонента между тестами и браузерными контейнерами. Основной задачей Grid является распределение тестовых сессий между доступными browser-node и управление удалённым выполнением тестов.

В рамках разработанного проекта browser-node запускаются внутри Docker-контейнеров. Использование Docker позволяет изолировать браузерные окружения, обеспечить воспроизводимость среды выполнения и упростить масштабирование тестовой инфраструктуры. Для выполнения тестов используются контейнеры с браузерами Google Chrome и Mozilla Firefox.

Каждый browser-container содержит:

- браузер;
- WebDriver;
- Selenium Node;
- необходимые системные зависимости.

После запуска контейнеры регистрируются в Selenium Grid и становятся доступны для выполнения тестовых сценариев.

Для формирования отчётности в проекте используется Allure Reports. После завершения выполнения тестов система автоматически формирует отчёты, содержащие:

- результаты выполнения тестов;
- информацию об ошибках;
- сведения о времени выполнения;
- статус прохождения тестовых сценариев;
- дополнительные данные о выполнении тестов.

Использование Allure позволяет упростить анализ результатов тестирования и повысить удобство сопровождения тестового проекта.

Таким образом, взаимодействие компонентов системы обеспечивает выполнение распределённого и параллельного UI-тестирования веб-

приложения, автоматизацию запуска тестов и централизованное формирование отчётности.

2.2.3. Структура проекта автоматизированного тестирования

Важным этапом проектирования фреймворка автоматизации тестирования является организация структуры проекта. Корректное разделение компонентов по пакетам повышает читаемость кода, упрощает сопровождение и обеспечивает масштабируемость системы..

Структура проекта представлена на рисунке 2.2.3.1.

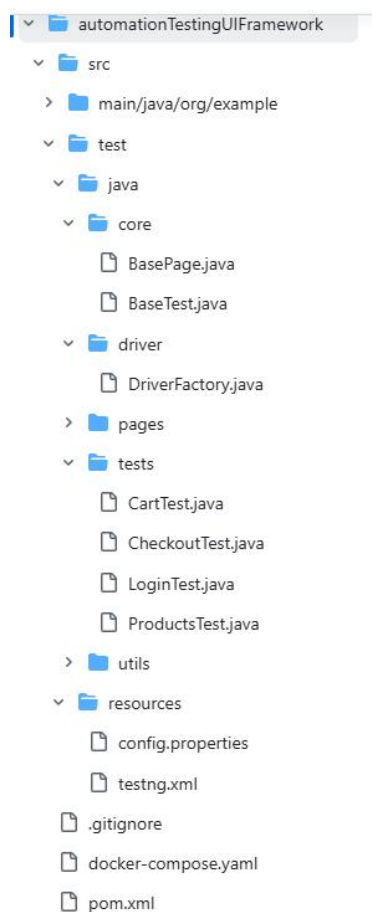


Рисунок 2.2.3.1. Структура проекта

Проект построен по модульному принципу с разделением на основные логические пакеты.

Пакет core содержит базовые классы фреймворка: BaseTest и BasePage. BasePage реализует общие методы взаимодействия с UI (клики, ввод текста, ожидания), а BaseTest отвечает за настройку тестового окружения, включая

инициализацию WebDriver и управление жизненным циклом тестов. Это позволяет централизовать общую логику и избежать дублирования кода.

Пакет driver включает класс DriverFactory, отвечающий за создание и конфигурацию WebDriver. Он обеспечивает поддержку локального и удалённого запуска браузеров через Selenium Grid, а также управление browser-session, изолируя работу с драйвером от тестовой логики.

Пакет pages реализует шаблон Page Object Model. Каждый класс соответствует отдельной странице веб-приложения и содержит локаторы элементов и методы взаимодействия с интерфейсом. В проекте реализованы страницы авторизации, каталога товаров, корзины и оформления заказа.

Пакет tests содержит тестовые сценарии, разделённые по функциональным областям: LoginTest, ProductsTest, CartTest, CheckoutTest. В них реализованы позитивные и негативные сценарии, а также проверки UI. Для выполнения тестов используется TestNG.

Пакет utils включает вспомогательные классы для работы с конфигурацией, генерацией тестовых данных и другими общими операциями, что снижает связанность компонентов и повышает переиспользуемость кода.

В каталоге resources находятся конфигурационные файлы: config.properties (настройки окружения, URL, параметры браузера, Selenium Grid) и testng.xml (наборы тестов, параметры параллельного выполнения и конфигурация запуска).

Файл pom.xml используется для управления зависимостями Maven и сборкой проекта. Через него подключены Selenium WebDriver, TestNG, Allure Reports и вспомогательные библиотеки, а также настроен процесс компиляции и запуска тестов.

Файл docker-compose.yml отвечает за развертывание тестовой инфраструктуры. С его помощью запускаются Selenium Hub и browser-node (Chrome и Firefox), что обеспечивает контейнеризированную и воспроизводимую среду выполнения тестов.

Таким образом, структура проекта построена по модульному принципу, что обеспечивает удобство сопровождения, масштабируемость и разделение ответственности между компонентами фреймворка. Общая UML-диаграмма классов представлена на рисунке ниже.

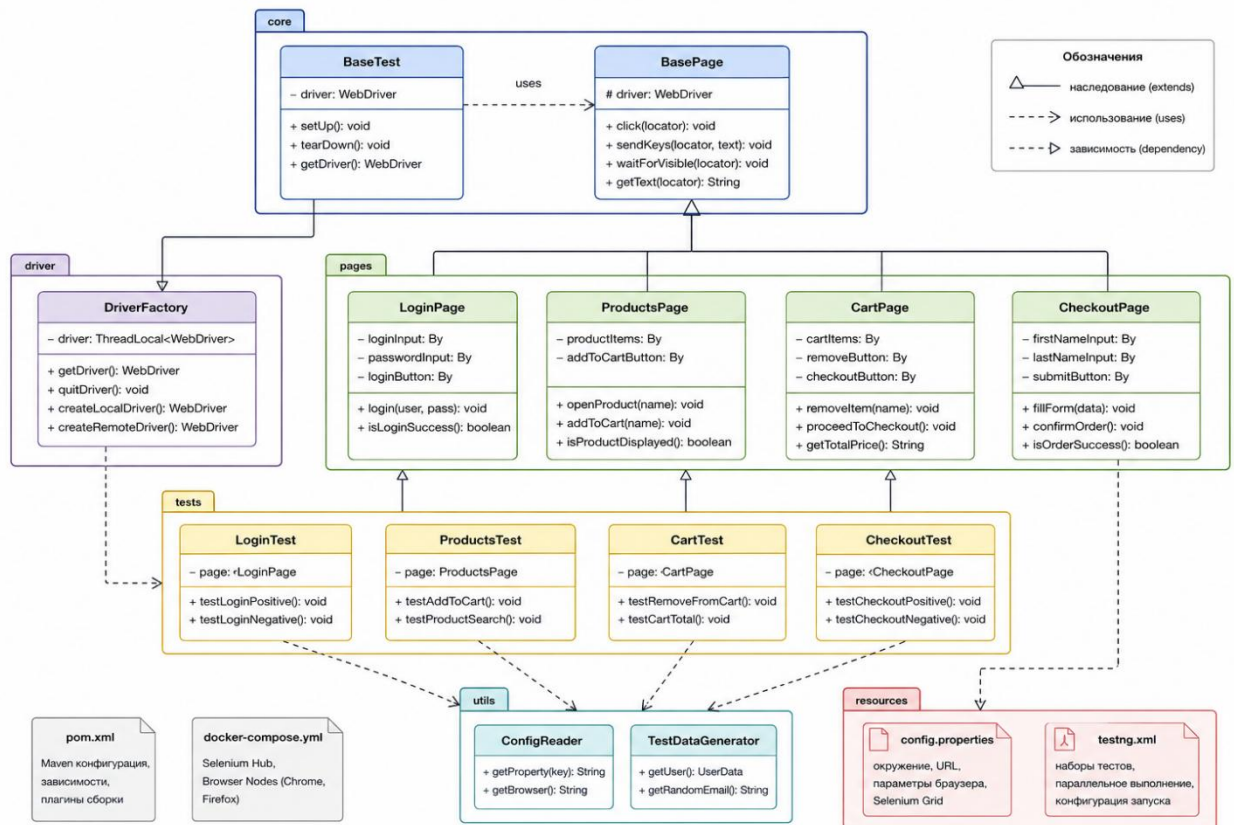


Рисунок 2.2.3.2. UML-диаграмма классов.

3. Проектирование распределённого выполнения тестов

2.3.1. Принципы работы Selenium Grid

При увеличении количества UI-тестов существенно возрастает время выполнения регрессионного тестирования, особенно при проверке различных браузеров и конфигураций. Для решения данной задачи в разработанном фреймворке используется Selenium Grid.

Selenium Grid — это компонент экосистемы Selenium, предназначенный для распределённого выполнения автоматизированных тестов. Он позволяет запускать тестовые сценарии параллельно на

нескольких браузерах и узлах инфраструктуры, что значительно сокращает общее время тестирования.

В рамках проекта Selenium Grid используется совместно с Docker-контейнерами, что обеспечивает масштабируемость и воспроизводимость тестовой среды.

Архитектура Selenium Grid включает центральный компонент Hub и набор исполнительных узлов Nodes.

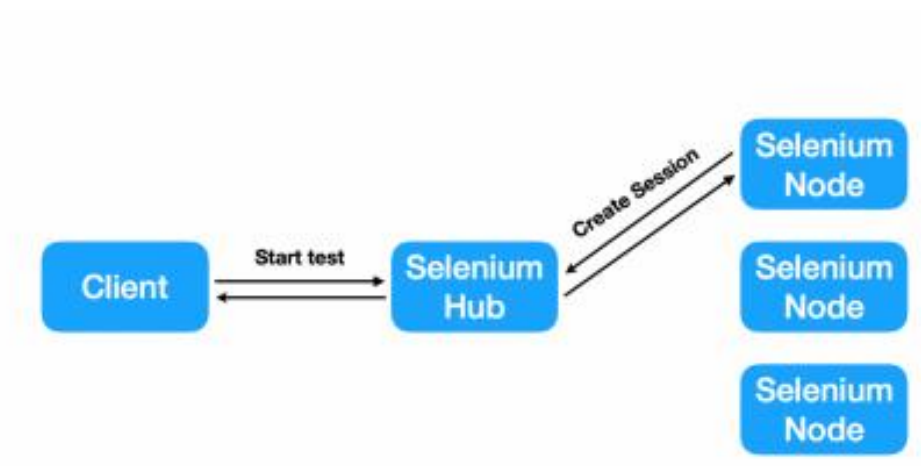


Рисунок 2.3.1. Архитектура Selenium Grid

Hub выполняет роль центра управления: принимает запросы на запуск тестов, распределяет их между узлами, управляет browser-session и маршрутизирует команды WebDriver.

Nodes представляют собой исполнительные окружения, в которых запускаются браузеры и выполняются тесты. В проекте они реализованы в виде Docker-контейнеров с браузерами Google Chrome и Mozilla Firefox и регистрируются в Hub.

Распределение тестов осуществляется автоматически: Selenium Hub направляет browser-session на доступные nodes с учётом типа браузера и загруженности узлов. Это позволяет выполнять тесты параллельно и эффективно использовать ресурсы системы.

Удалённое выполнение тестов обеспечивает запуск браузеров не на локальной машине, а на удалённых узлах Grid, что повышает масштабируемость, упрощает интеграцию с CI/CD и обеспечивает изоляцию окружения.

Для взаимодействия с Grid используется RemoteWebDriver, который передаёт команды через Selenium Hub к соответствующему node. Это позволяет запускать тесты независимо от конкретной инфраструктуры и обеспечивает гибкость масштабирования.

Таким образом, использование Selenium Grid и RemoteWebDriver позволяет реализовать распределённое и параллельное выполнение UI-тестов, повысить производительность и обеспечить масштабируемость тестовой инфраструктуры.

2.3.2. Использование Docker для контейнеризации среды тестирования

При разработке фреймворка одной из ключевых задач являлось обеспечение стабильной и воспроизводимой среды выполнения UI-тестов. Использование локальных окружений может приводить к различиям конфигураций и нестабильности тестов, поэтому в проекте применяется Docker.

Docker представляет собой платформу контейнеризации, позволяющую запускать изолированные окружения с заранее заданной конфигурацией. В рамках проекта он используется совместно с Selenium Grid для организации распределённого тестирования.

Браузеры в тестовой инфраструктуре запускаются внутри Docker-контейнеров, каждый из которых представляет собой Selenium Node с установленным браузером (Google Chrome или Mozilla Firefox), WebDriver и необходимыми зависимостями. После запуска контейнеры автоматически подключаются к Selenium Hub и используются для выполнения тестов, обеспечивая параллельный запуск сценариев.

Одним из основных преимуществ Docker является воспроизводимость среды. Все контейнеры используют одинаковые версии браузеров и драйверов, что исключает различия между локальными машинами и повышает стабильность тестов. Также это упрощает перенос и запуск проекта в различных окружениях, включая CI/CD.

Контейнеризация обеспечивает изоляцию тестовых запусков: каждая browser-session выполняется в отдельном контейнере, что исключает конфликты между тестами и повышает стабильность параллельного выполнения.

Для управления инфраструктурой используется Docker Compose. Файл docker-compose.yml описывает компоненты системы (Selenium Hub и browser-node) и позволяет запускать всю тестовую среду одной командой. Также он упрощает масштабирование Selenium Grid за счёт увеличения количества узлов без изменения архитектуры фреймворка.

Таким образом, использование Docker обеспечивает воспроизводимую, изолированную и масштабируемую среду автоматизированного тестирования, повышая стабильность и эффективность выполнения UI-тестов.

2.3.3. Проектирование параллельного запуска тестов

Одной из ключевых задач фреймворка является сокращение времени выполнения регрессионных тестов. При увеличении количества UI-тестов последовательный запуск становится неэффективным, поэтому в системе реализован параллельный запуск.

Параллельное выполнение тестов организовано с использованием возможностей TestNG, который поддерживает многопоточность через параметр parallel в testng.xml. В проекте используется запуск на уровне тестовых классов, что позволяет выполнять независимые сценарии одновременно в разных browser-session.

Количество одновременно выполняемых потоков задаётся параметром `thread-count`. Его значение зависит от количества доступных узлов Selenium Grid и ресурсов системы. Чрезмерное увеличение потоков может привести к перегрузке инфраструктуры и снижению стабильности, поэтому важно соблюдать баланс между скоростью и устойчивостью выполнения тестов.

При реализации параллельного запуска учитываются особенности многопоточности. Основной проблемой является совместное использование `WebDriver`, что может приводить к конфликтам `session` и нестабильному поведению тестов. Также важно исключить зависимости между тестовыми сценариями и обеспечить корректную очистку состояния.

Для решения этих проблем в фреймворке обеспечивается полная независимость тестов: каждый сценарий выполняется в собственной `browser-session` и не зависит от других тестов. Это повышает стабильность и позволяет безопасно использовать параллельный запуск.

Для потокобезопасного управления `WebDriver` применяется `ThreadLocal`, который хранит отдельный экземпляр драйвера для каждого потока. Это обеспечивает изоляцию браузерных сессий и предотвращает конфликты между тестами. Управление драйвером централизовано через `DriverFactory`.

Использование параллельного запуска совместно с Selenium Grid позволяет распределять тесты между несколькими узлами, выполнять их одновременно в разных браузерах и значительно сокращать общее время тестирования.

Таким образом, параллельное выполнение тестов повышает производительность фреймворка и обеспечивает эффективное использование ресурсов тестовой инфраструктуры.

4. Проектирование системы отчётности и интеграции

2.4.1. Организация формирования отчётов о тестировании

Одним из важных компонентов системы автоматизированного тестирования является механизм формирования отчётности. Наличие подробных отчётов позволяет анализировать результаты выполнения тестов, выявлять ошибки, контролировать стабильность тестового набора и упрощать сопровождение проекта. Без централизованной системы отчётности процесс анализа результатов тестирования становится значительно сложнее, особенно при параллельном и распределённом выполнении большого количества тестовых сценариев.

В рамках разработанного фреймворка для формирования отчётов используется система Allure Reports. Данный инструмент интегрирован с TestNG и позволяет автоматически собирать информацию о выполнении тестовых сценариев, формируя структурированные отчёты после завершения тестирования. Использование Allure обеспечивает наглядное отображение результатов тестирования и упрощает процесс анализа состояния тестового проекта.

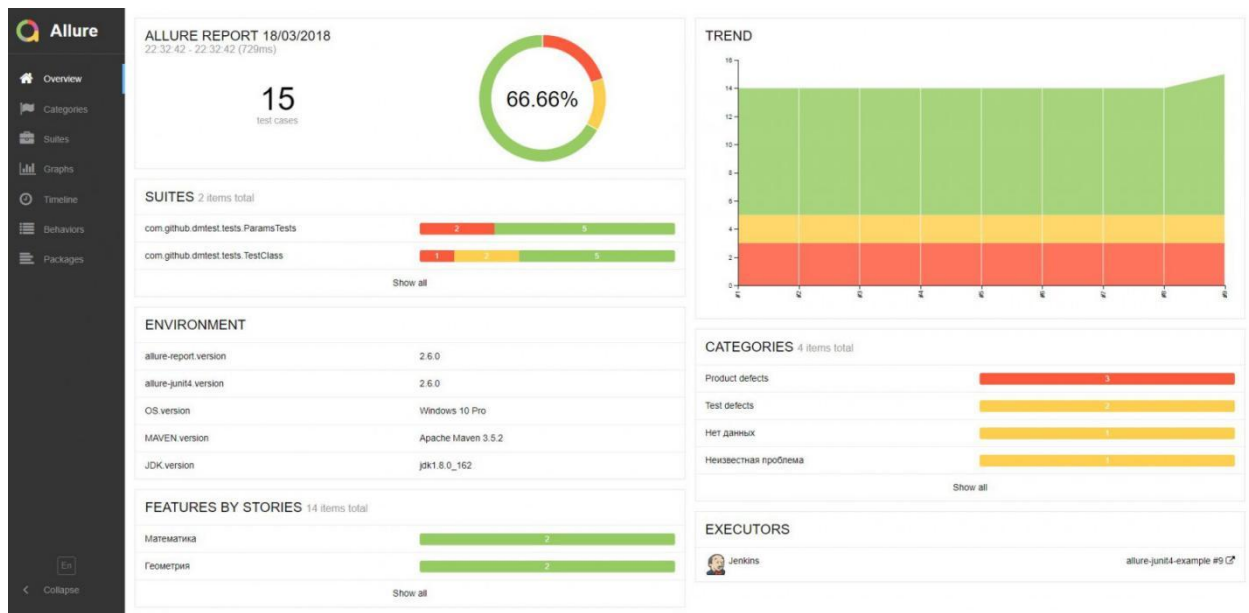


Рисунок 2.4.1. Пример Allure отчета

Сформированные отчёты содержат информацию о выполненных тестах, времени их выполнения, статусе прохождения сценариев и возникающих ошибках. Дополнительно отчёты позволяют отслеживать последовательность действий, выполненных во время тестирования, что особенно важно при анализе нестабильных UI-тестов.

Для повышения информативности отчётности в проекте используется разбиение тестовых сценариев на отдельные шаги. Каждый тест включает последовательность действий пользователя, таких как открытие страницы, ввод данных, нажатие элементов интерфейса и выполнение проверок. Отображение шагов в отчётах позволяет определить этап, на котором возникла ошибка, и существенно упрощает диагностику проблем.

Дополнительно в разработанном фреймворке реализовано автоматическое создание скриншотов при возникновении ошибок во время выполнения тестов. В случае падения тестового сценария текущее состояние пользовательского интерфейса сохраняется в виде изображения и прикрепляется к отчёту Allure. Использование скриншотов позволяет быстрее определить причину возникновения ошибки и упрощает анализ поведения приложения в момент сбоя.

Для получения дополнительной информации о процессе выполнения тестов в системе реализован механизм логирования ошибок. Во время выполнения сценариев фиксируются возникающие исключения, ошибки Selenium WebDriver и другие события, связанные с работой тестовой инфраструктуры. Логирование позволяет получать техническую информацию о состоянии системы и повышает удобство сопровождения тестового проекта.

Результаты выполнения тестов сохраняются в отдельных каталогах проекта и могут использоваться для последующего анализа. Хранение отчётов, логов и скриншотов позволяет отслеживать историю запусков, анализировать стабильность тестов и сравнивать результаты регрессионного тестирования. Кроме того, сохранённые результаты могут использоваться в

CI/CD-системах для автоматического контроля качества программного продукта после выполнения тестовых запусков.

Таким образом, реализованная система формирования отчётности обеспечивает централизованный механизм анализа результатов автоматизированного тестирования, повышает удобство диагностики ошибок и упрощает сопровождение фреймворка автоматизации UI-тестирования.

2.4.2. Сценарий работы разработанного фреймворка

Процесс работы разработанного фреймворка автоматизированного тестирования представляет собой последовательность взаимосвязанных этапов, обеспечивающих запуск UI-тестов, выполнение тестовых сценариев в распределённой среде и формирование отчётности. Общая схема работы фреймворка представлена на рисунке 2.4.2.

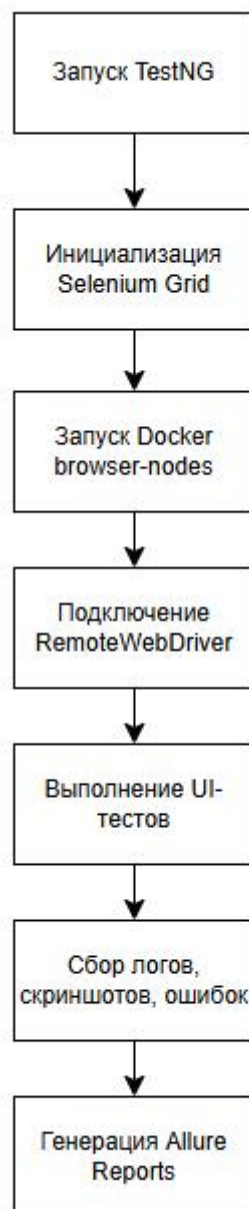


Рисунок 2.4.2. Сценарий работы фреймворка автоматизированного тестирования

На первом этапе выполняется запуск тестов с использованием фреймворка TestNG. Запуск может осуществляться как локально, так и в рамках CI/CD-процесса. Конфигурация тестового запуска определяется параметрами файла `testng.xml`, в котором задаются наборы тестов, режим параллельного выполнения и количество потоков.

После запуска тестов выполняется инициализация Selenium Grid. На данном этапе осуществляется запуск Selenium Hub и регистрация browser-

node, функционирующих внутри Docker-контейнеров. Grid подготавливает инфраструктуру для распределённого выполнения тестовых сценариев.

Далее тестовые сценарии подключаются к Selenium Grid с использованием RemoteWebDriver. В зависимости от конфигурации тестов Selenium Grid распределяет browser-session между доступными browser-node. В рамках разработанного проекта используются контейнеры с браузерами Google Chrome и Mozilla Firefox.

После создания browser-session начинается выполнение UI-тестов. Selenium WebDriver выполняет пользовательские действия в браузере, включая:

- открытие страниц;
- ввод данных;
- взаимодействие с элементами интерфейса;
- выполнение проверок.

Во время выполнения тестов система автоматически собирает результаты выполнения сценариев, информацию об ошибках, логи и скриншоты. При возникновении исключений состояние пользовательского интерфейса фиксируется и сохраняется для последующего анализа.

После завершения тестирования выполняется генерация отчётов Allure Reports. Система формирует структурированный отчёт, содержащий информацию о результатах выполнения тестов, времени выполнения сценариев, возникших ошибках и статусе прохождения тестов.

Таким образом, разработанный фреймворк обеспечивает полный цикл автоматизированного UI-тестирования, включая подготовку инфраструктуры, распределённое выполнение тестов и централизованное формирование отчётности.

5. Выводы по второй главе

2.5.1. Основные результаты проектирования

В рамках второй главы было выполнено проектирование фреймворка автоматизации UI-тестирования веб-приложений с использованием технологий Java, Selenium WebDriver, TestNG, Selenium Grid и Docker. В ходе проектирования были определены основные архитектурные и инфраструктурные решения, обеспечивающие стабильность, масштабируемость и удобство сопровождения системы автоматизированного тестирования.

В процессе разработки архитектуры фреймворка был выбран подход Page Object Model (POM), обеспечивающий разделение тестовой логики и логики взаимодействия с пользовательским интерфейсом. Использование данного подхода позволило уменьшить дублирование кода, повысить читаемость тестовых сценариев и упростить сопровождение тестового проекта.

Дополнительно была спроектирована структура тестового проекта, включающая отдельные пакеты для тестов, Page Object компонентов, базовых классов, драйверов, вспомогательных компонентов и конфигурационных файлов. Подобная организация проекта обеспечивает модульность системы и возможность дальнейшего расширения тестового набора.

В рамках проектирования инфраструктуры автоматизированного тестирования была реализована интеграция Selenium Grid и Docker. Использование распределённой архитектуры позволяет выполнять UI-тесты удалённо и запускать браузеры внутри контейнеризированной среды. Контейнеризация обеспечивает воспроизводимость окружения, изоляцию тестовых сессий и упрощает масштабирование тестовой инфраструктуры.

Особое внимание было уделено организации параллельного выполнения тестов. В проекте реализованы механизмы parallel execution на основе возможностей TestNG и Selenium Grid. Для обеспечения

потокбезопасной работы WebDriver был использован механизм ThreadLocal, позволяющий создавать отдельные browser-session для каждого потока выполнения.

Также была спроектирована система формирования отчётности с использованием Allure Reports. Реализованный механизм позволяет автоматически собирать результаты выполнения тестов, сохранять логи и скриншоты, а также формировать структурированные отчёты для анализа результатов тестирования.

Таким образом, в результате проектирования была разработана архитектура масштабируемого фреймворка автоматизации UI-тестирования, обеспечивающего поддержку распределённого и параллельного выполнения тестов, централизованное управление тестовой инфраструктурой и автоматизированное формирование отчётности.

2.5.2. Подготовка к реализации фреймворка

В результате выполнения проектирования были определены основные компоненты и архитектурные решения, необходимые для реализации фреймворка автоматизации UI-тестирования. Разработанная структура системы позволяет перейти к практической реализации тестовой инфраструктуры и созданию набора автоматизированных тестов.

На основе выбранной архитектуры будут реализованы базовые классы фреймворка, включая компоненты для управления WebDriver, организации тестовых сценариев и взаимодействия со страницами веб-приложения. Дополнительно будет выполнена реализация Page Object классов для основных модулей тестируемого веб-приложения.

Следующим этапом станет настройка распределённой инфраструктуры Selenium Grid с использованием Docker-контейнеров. Будет выполнено развёртывание Selenium Hub и browser-node, обеспечивающих удалённое и параллельное выполнение UI-тестов в различных браузерах.

После настройки инфраструктуры будет реализован набор автоматизированных тестовых сценариев для проверки функциональности веб-приложения SauceDemo. В рамках тестирования планируется реализация позитивных и негативных тестов для модулей авторизации, каталога товаров, корзины и оформления заказа.

Дополнительно будет выполнена интеграция системы отчётности Allure Reports, позволяющей формировать структурированные отчёты о результатах выполнения тестов, сохранять информацию об ошибках и прикреплять скриншоты при возникновении сбоев.

Таким образом, разработанные архитектурные решения создают основу для практической реализации фреймворка автоматизации тестирования. В следующей главе будет рассмотрен процесс реализации компонентов системы, настройки тестовой инфраструктуры и демонстрация работы разработанного фреймворка автоматизированного UI-тестирования.

Глава 3. Реализация фреймворка автоматизации UI-тестирования веб-приложений

1. Реализация архитектуры и инфраструктуры фреймворка

3.1.1. Настройка проекта и используемых технологий

Для реализации фреймворка автоматизации UI-тестирования был выбран язык программирования Java и система сборки Maven. Использование Java обусловлено широкой поддержкой инструментов автоматизированного тестирования, развитой экосистемой библиотек и удобством интеграции с Selenium WebDriver и TestNG.

В качестве системы управления зависимостями и сборки проекта использовался Maven. Данный инструмент позволяет централизованно управлять библиотеками проекта, автоматизировать процесс сборки и упрощает интеграцию фреймворка с CI/CD-системами.

Основные зависимости проекта подключались через конфигурационный файл `pom.xml`. В рамках разработанного фреймворка были использованы следующие технологии:

- Selenium WebDriver — для автоматизации взаимодействия с веб-браузером;
- TestNG — для организации тестовых сценариев и управления выполнением тестов;
- Allure Reports — для формирования отчётности;
- WebDriverManager — для автоматического управления драйверами браузеров.

Для реализации UI-тестирования использовалась библиотека Selenium WebDriver. Данный инструмент обеспечивает взаимодействие с элементами пользовательского интерфейса веб-приложения и позволяет выполнять автоматизированные действия в браузере, имитируя поведение пользователя.

Организация выполнения тестовых сценариев осуществлялась с использованием TestNG. Данный фреймворк предоставляет возможности группировки тестов, настройки параллельного выполнения, управления

жизненным циклом тестирования и формирования конфигурации тестовых запусков через файл testng.xml.

Для формирования отчётов о результатах тестирования была интегрирована система Allure Reports. Использование Allure позволяет автоматически собирать результаты выполнения тестов, отображать шаги тестирования, прикреплять скриншоты и формировать структурированные HTML-отчёты.

Конфигурация зависимостей проекта реализована в файле pom.xml. Пример используемых зависимостей представлен в листинге 3.1.

Листинг 3.1 — Конфигурация зависимостей Maven в файле pom.xml

```
<dependencies>
  <!-- Source: https://mvnrepository.com/artifact/io.github.bonigarcia/webdrivermanager -->
  <dependency>
    <groupId>io.github.bonigarcia</groupId>
    <artifactId>webdrivermanager</artifactId>
    <version>6.3.3</version>
    <scope>compile</scope>
  </dependency>
  <!-- Source: https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java -->
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.41.0</version>
    <scope>compile</scope>
  </dependency>
  <!-- Source: https://mvnrepository.com/artifact/org.testng/testng -->
  <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>7.12.0</version>
    <scope>test</scope>
  </dependency>
  <!-- Source: https://mvnrepository.com/artifact/io.qameta.allure/allure-testng -->
  <dependency>
    <groupId>io.qameta.allure</groupId>
    <artifactId>allure-testng</artifactId>
    <version>2.33.0</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Таким образом, выбранный стек технологий обеспечивает реализацию масштабируемого фреймворка автоматизации UI-тестирования, поддерживающего распределённое выполнение тестов, параллельный запуск и автоматическое формирование отчётности.

3.1.2. Реализация архитектуры фреймворка

Архитектура разработанного фреймворка автоматизации тестирования построена на основе паттерна Page Object Model (POM). Использование данного подхода позволило разделить логику тестовых сценариев и логику взаимодействия с пользовательским интерфейсом веб-приложения. Подобная организация проекта повышает читаемость тестов, уменьшает дублирование кода и упрощает сопровождение тестового набора.

В рамках разработанного фреймворка архитектура системы включает несколько основных компонентов:

- BaseTest — базовый класс для организации жизненного цикла тестов;
- BasePage — базовый класс для Page Object компонентов;
- DriverFactory — компонент управления экземплярами WebDriver;
- Page Object классы — классы страниц веб-приложения;
- тестовые классы — реализация тестовых сценариев.

Общая структура взаимодействия компонентов представлена на рисунке 3.1.2.

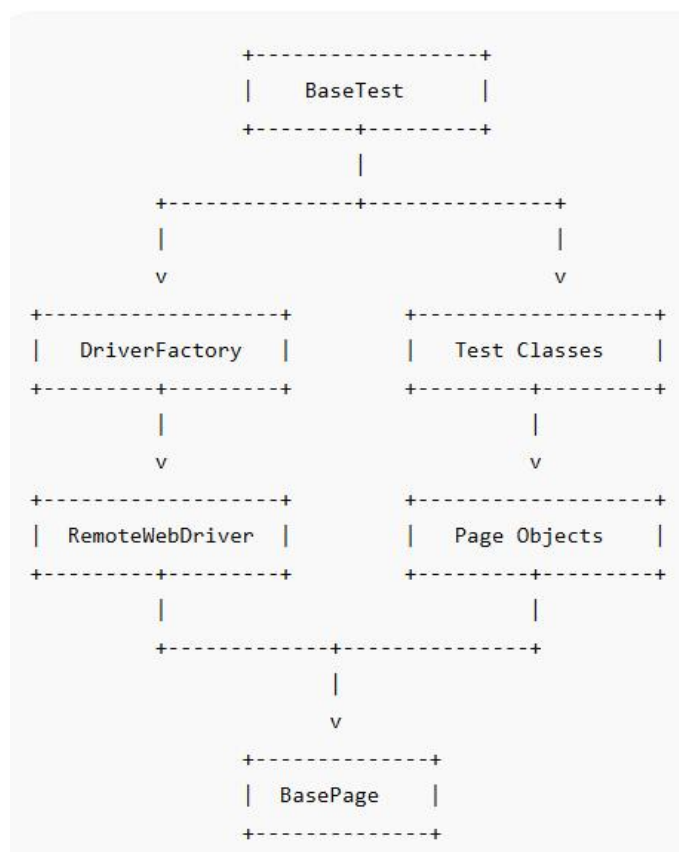


Рисунок 3.1.2. Архитектура основных компонентов фреймворка

Базовым компонентом системы является класс `BaseTest`, отвечающий за подготовку и завершение тестового окружения. В данном классе реализована инициализация `WebDriver` перед запуском тестов, а также закрытие `browser-session` после завершения выполнения сценариев.

Для управления экземплярами браузеров используется класс `DriverFactory`. Данный компонент реализует создание локальных и удалённых экземпляров `WebDriver`, а также обеспечивает подключение к `Selenium Grid` через `RemoteWebDriver`.

Одной из важных особенностей реализации является поддержка многопоточности. Для обеспечения независимой работы браузеров при параллельном выполнении тестов используется механизм `ThreadLocal`. Это позволяет создавать отдельный экземпляр `WebDriver` для каждого потока выполнения и предотвращает конфликты между тестовыми сценариями.

Пример реализации хранения экземпляра драйвера представлен в листинге 3.2.

Листинг 3.2 — Использование `ThreadLocal` для хранения `WebDriver`

```
private static ThreadLocal<WebDriver> driver = new ThreadLocal<>();  
public static WebDriver getDriver() {  
    return driver.get();  
}
```

Для реализации взаимодействия со страницами веб-приложения использовался паттерн `Page Object Model`. Каждая страница приложения представлена отдельным классом, содержащим:

- локаторы элементов;
- методы взаимодействия со страницей;
- проверки состояния интерфейса.

Все `Page Object` классы наследуются от `BasePage`, содержащего общие методы работы с браузером и элементами пользовательского интерфейса.

Использование базового класса позволяет централизовать общую функциональность, включая:

- ожидание элементов;

- выполнение кликов;
- ввод текста;
- получение данных со страницы.

Тестовые классы содержат исключительно логику тестовых сценариев и не работают напрямую с локаторами элементов интерфейса. Взаимодействие с приложением осуществляется через методы Page Object компонентов, что повышает читаемость и поддерживаемость тестового кода.

Таким образом, реализованная архитектура фреймворка обеспечивает модульность, масштабируемость и поддержку параллельного выполнения тестов, а также упрощает сопровождение и расширение тестового проекта.

3.1.3. Реализация распределённой инфраструктуры тестирования

Для обеспечения распределённого и параллельного выполнения UI-тестов в разработанном фреймворке была реализована инфраструктура на основе Selenium Grid и Docker. Использование данной архитектуры позволяет запускать тестовые сценарии одновременно в нескольких браузерах и изолированных контейнерах, что существенно сокращает общее время выполнения тестового набора.

В качестве основы распределённой инфраструктуры использовался Selenium Grid, обеспечивающий централизованное управление browser-session и распределение тестов между доступными browser-node. Тестовые сценарии подключаются к Grid через RemoteWebDriver, после чего Selenium Hub перенаправляет выполнение тестов в соответствующие контейнеры браузеров.

Общая схема распределённой инфраструктуры представлена на рисунке 3.1.3.1.

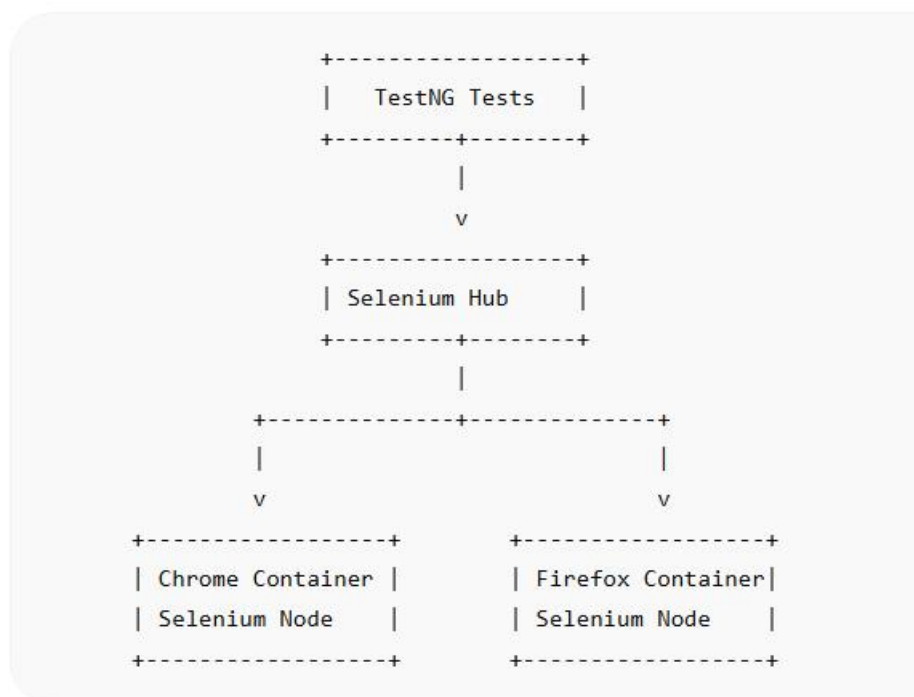


Рисунок 3.1.3.1. Распределённая инфраструктура Selenium Grid

Для контейнеризации тестовой среды использовалась технология Docker. Каждый browser-node запускался внутри отдельного контейнера, содержащего браузер и Selenium Node. Подобный подход обеспечивает воспроизводимость среды тестирования и позволяет изолировать browser-session друг от друга.

Развёртывание инфраструктуры выполнялось с использованием файла `docker-compose.yml`, обеспечивающего централизованный запуск Selenium Hub и browser-node. Использование Docker Compose существенно упрощает управление тестовой инфраструктурой и позволяет быстро масштабировать количество контейнеров при необходимости.

Пример конфигурации `docker-compose.yml` представлен в листинге 3.3.

Листинг 3.3 — Конфигурация Selenium Grid в `docker-compose.yml`

```

version: "3"
services:
  selenium-hub:
    image: selenium/hub:4.21.0
    container_name: selenium-hub
    ports:
      - "4444:4444"
  chrome:
    image: selenium/node-chrome:4.21.0
    shm_size: 2gb
    depends_on:

```

```
- selenium-hub
environment:
- SE_EVENT_BUS_HOST=selenium-hub
- SE_EVENT_BUS_PUBLISH_PORT=4442
- SE_EVENT_BUS_SUBSCRIBE_PORT=4443
firefox:
image: selenium/node-firefox:4.21.0
shm_size: 2gb
depends_on:
- selenium-hub
environment:
- SE_EVENT_BUS_HOST=selenium-hub
- SE_EVENT_BUS_PUBLISH_PORT=4442
- SE_EVENT_BUS_SUBSCRIBE_PORT=4443
```

Для повышения производительности тестирования во фреймворке была реализована поддержка параллельного выполнения тестов средствами TestNG. Настройка многопоточности выполнялась через файл `testng.xml`, в котором задавались параметры `parallel` и `thread-count`. Использование параллельного запуска позволяет одновременно выполнять несколько тестовых сценариев в разных `browser-session`.

При выполнении тестов Selenium Grid автоматически распределяет потоки между доступными `browser-node`. Благодаря использованию `ThreadLocal` каждый поток получает независимый экземпляр `WebDriver`, что предотвращает конфликты при многопоточном выполнении.

Дополнительно в процессе разработки выполнялось тестирование работы Selenium Grid через встроенный веб-интерфейс Grid UI, позволяющий отслеживать активные `browser-session` и состояние подключённых `browser-node`. Пример интерфейса Selenium Grid может быть представлен на рисунке 3.3.

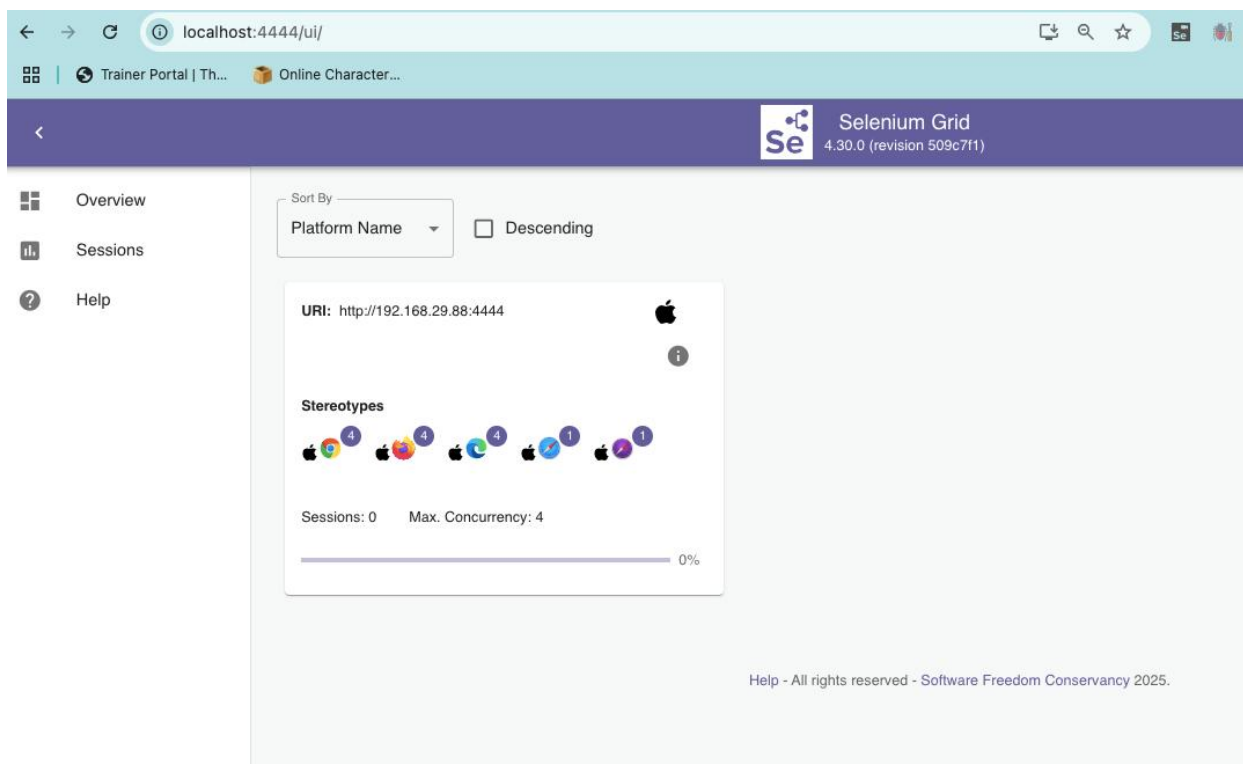


Рисунок 3.1.3.2. Веб-интерфейс Selenium Grid

В результате реализации распределённой инфраструктуры был создан механизм параллельного выполнения UI-тестов в контейнеризированной среде, обеспечивающий масштабируемость, воспроизводимость тестового окружения и сокращение времени выполнения тестовых сценариев.

2. Реализация автоматизированных UI-тестов

3.2.1. Реализация тестовых сценариев

В рамках разработанного фреймворка был реализован набор автоматизированных UI-тестов для веб-приложения SauceDemo. Тестовые сценарии охватывают основные функциональные модули приложения, включая авторизацию пользователей, работу с каталогом товаров, корзиной и оформлением заказа.

Реализация тестов выполнялась с использованием Selenium WebDriver и TestNG. Все тестовые сценарии организованы в отдельные тестовые классы в соответствии с функциональными модулями приложения. Подобная структура упрощает сопровождение тестового набора и обеспечивает логическое разделение тестируемой функциональности.

Для проверки модуля авторизации был реализован класс `LoginTest`. В рамках данного класса выполняются проверки успешной авторизации, обработки некорректных учётных данных, отображения сообщений об ошибках и поведения системы при попытке входа заблокированного пользователя.

Для тестирования каталога товаров был реализован класс `ProductsTest`. В данных тестах проверяется отображение списка товаров, наличие названий и цен, а также корректность работы механизмов сортировки товаров по цене и названию.

Работа корзины реализована в классе `CartTest`. В процессе тестирования выполняются проверки добавления товаров в корзину, удаления товаров, корректности отображения счётчика корзины и сохранения выбранных товаров при переходе между страницами приложения.

Дополнительно был реализован класс `CheckoutTest`, предназначенный для проверки сценариев оформления заказа. Данные тесты включают проверку заполнения формы оформления, обработку незаполненных обязательных полей и успешное завершение оформления заказа.

Основные реализованные тестовые сценарии представлены в таблице 3.2.1.

Таблица 3.1 — Основные автоматизированные UI-тесты

Тестовый класс	Проверяемая функциональность
<code>LoginTest</code>	Авторизация пользователей
<code>ProductsTest</code>	Работа каталога товаров
<code>CartTest</code>	Работа корзины
<code>CheckoutTest</code>	Оформление заказа

Тестовые сценарии реализованы с использованием Page Object Model, благодаря чему тестовые классы содержат исключительно бизнес-логику

проверок и не взаимодействуют напрямую с локами элементов интерфейса.

Пример реализации теста успешной авторизации представлен в листинге 3.4.

Листинг 3.4 — Тест успешной авторизации

```
@Test
public void testSuccessfulLogin() {
    LoginPage loginPage = new LoginPage(driver);
    loginPage.enterUsername("standard_user");
    loginPage.enterPassword("secret_sauce");
    loginPage.clickLoginButton();
    Assert.assertTrue(productsPage.isProductsPageDisplayed());
}
```

Для повышения читаемости тестов взаимодействие с пользовательским интерфейсом вынесено в отдельные Page Object классы. Пример метода Page Object компонента представлен в листинге 3.5.

Листинг 3.5 — Метод Page Object компонента

```
public void clickLoginButton() {
    loginButton.click();
}
```

В рамках тестирования активно использовались проверки (Assert) для контроля корректности работы приложения. Проверялись URL страниц, наличие элементов интерфейса, отображение сообщений об ошибках и корректность отображаемых данных.

Дополнительно в тестовых сценариях использовались механизмы параллельного выполнения TestNG, что позволило одновременно запускать несколько тестов в различных browser-session Selenium Grid.

Таким образом, реализованный набор автоматизированных UI-тестов обеспечивает проверку ключевой функциональности веб-приложения и демонстрирует работу разработанного фреймворка автоматизации тестирования.

3.2.2. Реализация Page Object компонентов

Для организации взаимодействия с пользовательским интерфейсом веб-приложения в разработанном фреймворке использовался паттерн Page Object Model. В рамках данного подхода каждая страница приложения представлена отдельным классом, содержащим локаторы элементов интерфейса и методы взаимодействия с ними.

В проекте были реализованы основные Page Object компоненты:

- LoginPage;
- ProductsPage;
- CartPage;
- CheckoutPage.

Каждый Page Object класс инкапсулирует работу с элементами конкретной страницы и предоставляет набор методов, используемых тестовыми сценариями. Подобный подход позволяет отделить тестовую логику от деталей реализации пользовательского интерфейса и уменьшает дублирование кода.

Класс LoginPage реализует взаимодействие со страницей авторизации и содержит методы для ввода логина, пароля и выполнения авторизации пользователя. В классе используются локаторы полей ввода и кнопки входа.

Пример реализации локаторов и методов страницы авторизации представлен в листинге 3.2.2.

Листинг 3.2.2 — Реализация класса LoginPage

```
@FindBy(id = "user-name")
private WebElement usernameInput;
@FindBy(id = "password")
private WebElement passwordInput;
@FindBy(id = "login-button")
private WebElement loginButton;
public void enterUsername(String username) {
    usernameInput.sendKeys(username);
}
public void enterPassword(String password) {
    passwordInput.sendKeys(password);
}
public void clickLoginButton() {
    loginButton.click();
}
```

Класс ProductsPage предназначен для взаимодействия со страницей каталога товаров. В данном компоненте реализованы методы получения

списка товаров, работы с сортировкой и проверки отображения элементов каталога.

Класс CartPage обеспечивает взаимодействие с корзиной пользователя. В рамках данного компонента реализованы методы добавления и удаления товаров, получения количества товаров в корзине и проверки содержимого корзины.

Общая схема взаимодействия тестовых классов и Page Object компонентов представлена на рисунке 3.2.2.

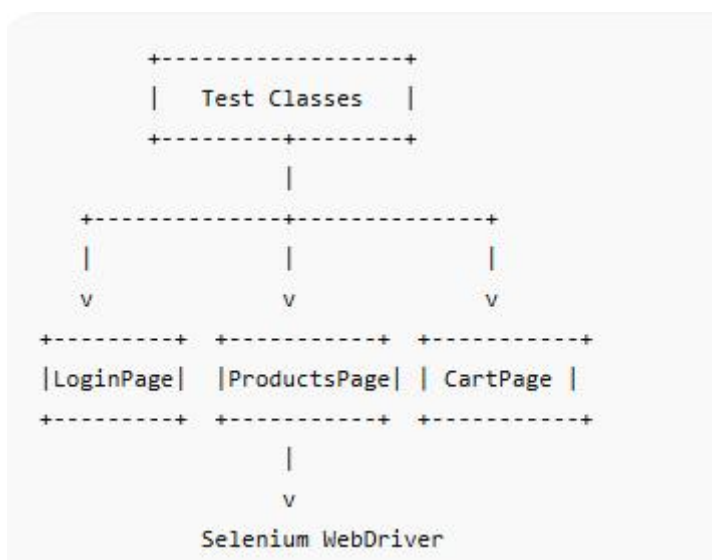


Рисунок 3.2.2. Взаимодействие тестов и Page Object компонентов

Использование Page Object Model позволило централизовать работу с элементами интерфейса и повысить удобство сопровождения тестового проекта. При изменении структуры пользовательского интерфейса модификации вносятся только в соответствующий Page Object класс, что уменьшает количество изменений в тестовых сценариях.

3. Формирование отчётности и анализ результатов тестирования

3.3.1. Интеграция Allure Reports

Для формирования отчётности о результатах выполнения автоматизированных UI-тестов в разработанном фреймворке использовалась система Allure Reports. Интеграция Allure с TestNG позволила автоматически собирать результаты тестовых запусков и формировать структурированные HTML-отчёты.

Подключение Allure выполнялось через Maven-зависимости, указанные в файле `pom.xml`, а генерация отчётов осуществлялась после завершения выполнения тестового набора. Использование Allure Reports обеспечивает удобное отображение результатов тестирования и упрощает анализ состояния тестового проекта.

Сформированные отчёты содержат информацию о статусе выполнения тестов, времени выполнения сценариев, количестве успешных и неуспешных тестов, а также сведения о возникших ошибках. Дополнительно Allure позволяет отображать последовательность действий, выполненных во время тестирования.

Для повышения информативности отчётности в тестовых сценариях использовались шаги тестирования (steps). Каждый этап взаимодействия с пользовательским интерфейсом фиксировался в отчёте, что позволяет определить последовательность действий пользователя и упростить анализ причин возникновения ошибок.

Пример использования шага тестирования представлен в листинге 3.3.1.

Листинг 3.3.1. — Использование шагов Allure

```
@Step("Ввод логина пользователя")
public void enterUsername(String username) {
    usernameInput.sendKeys(username);
}
```

Дополнительно во фреймворке была реализована автоматическая генерация скриншотов при возникновении ошибок во время выполнения тестов. В случае падения тестового сценария текущее состояние браузера сохраняется и прикрепляется к отчёту Allure. Использование скриншотов позволяет визуально определить причину возникновения ошибки и ускоряет процесс анализа дефектов.

Также в процессе выполнения тестов осуществлялось логирование ошибок и событий тестирования. Информация о возникающих исключениях и ошибках Selenium WebDriver сохранялась в логах и использовалась для диагностики нестабильных тестов.

Пример интерфейса сформированного отчёта Allure представлен на рисунке 3.3.1.

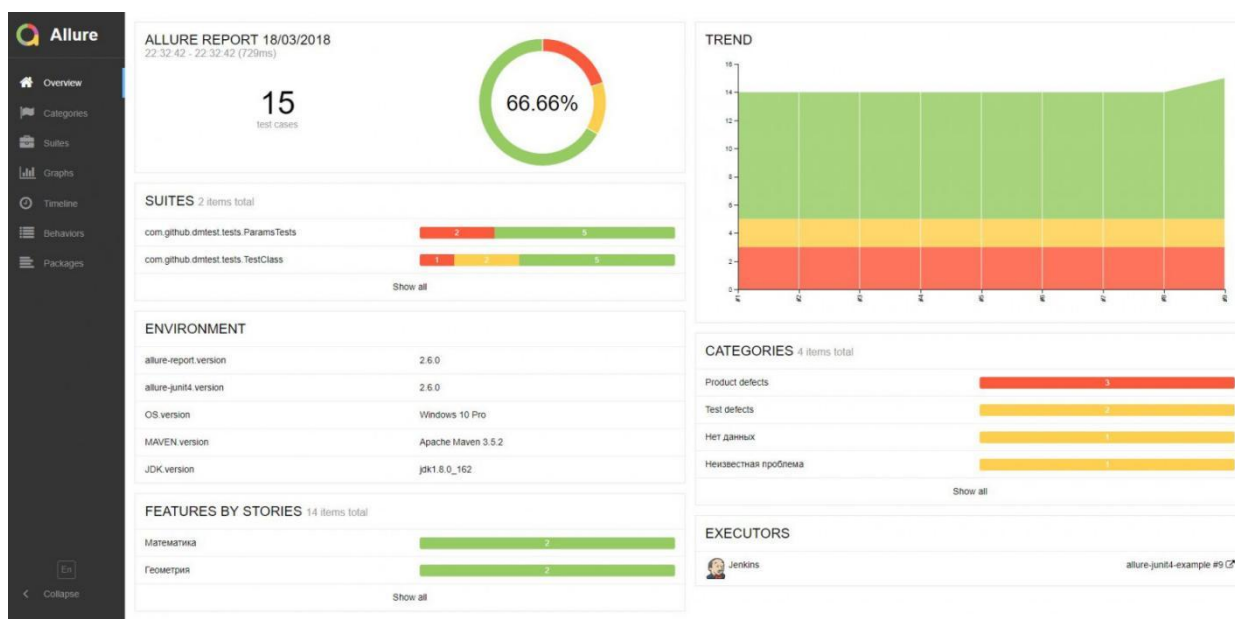


Рисунок 3.3.1.Отчёт Allure Reports

Отчёты Allure содержат сведения о выполненных тестах, отображают шаги выполнения сценариев, прикрепленные скриншоты и информацию об ошибках. Подобный подход существенно упрощает анализ результатов тестирования и повышает удобство сопровождения тестового проекта.

Таким образом, интеграция Allure Reports позволила реализовать централизованную систему формирования отчётности, обеспечивающую наглядное представление результатов автоматизированного тестирования и упрощающую диагностику ошибок.

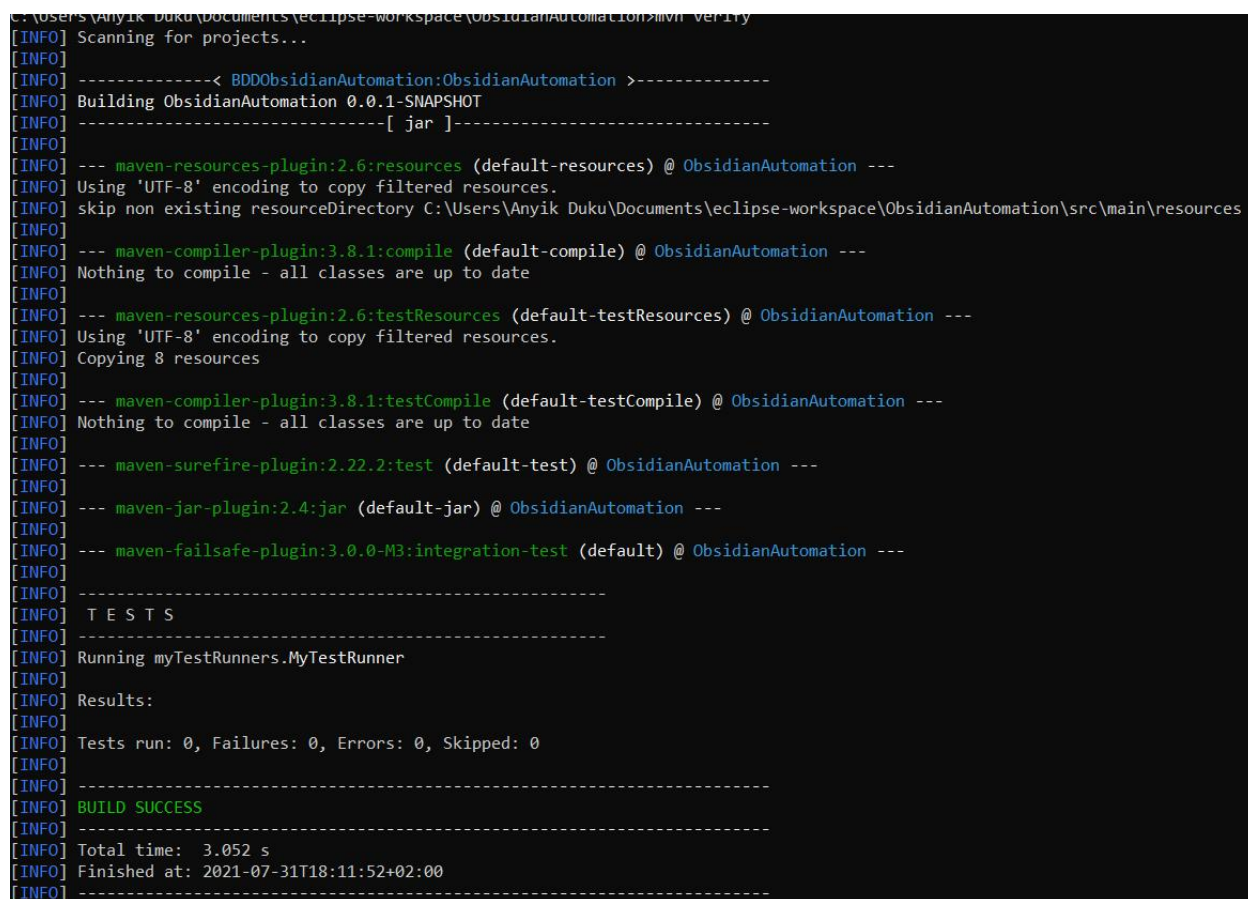
3.3.2. Анализ результатов выполнения тестов

После реализации фреймворка автоматизации тестирования было выполнено тестирование работы системы в условиях последовательного и параллельного выполнения UI-тестов. Основной целью анализа являлась проверка стабильности работы тестовой инфраструктуры, корректности распределённого выполнения тестов и оценка производительности разработанного решения.

Запуск тестового набора осуществлялся с использованием TestNG и Selenium Grid. Во время выполнения тестов Selenium Grid распределял browser-session между доступными browser-node, функционирующими внутри Docker-контейнеров. Использование распределённой инфраструктуры позволило одновременно выполнять несколько тестовых сценариев в различных браузерах.

В процессе тестирования выполнялись сценарии авторизации, работы с каталогом товаров, корзиной и оформления заказа. Результаты выполнения тестов отображались как в консоли Maven/TestNG, так и в системе отчётности Allure Reports.

Пример выполнения тестов в консоли представлен на рисунке 3.3.2.



```
C:\Users\Anyik Duku\Documents\eclipse-workspace\ObsidianAutomation>mvn verify
[INFO] Scanning for projects...
[INFO]
[INFO] -----< BDDObsidianAutomation:ObsidianAutomation >-----
[INFO] Building ObsidianAutomation 0.0.1-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ ObsidianAutomation ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] skip non existing resourceDirectory C:\Users\Anyik Duku\Documents\eclipse-workspace\ObsidianAutomation\src\main\resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ ObsidianAutomation ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ ObsidianAutomation ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] Copying 8 resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:testCompile (default-testCompile) @ ObsidianAutomation ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- maven-surefire-plugin:2.22.2:test (default-test) @ ObsidianAutomation ---
[INFO]
[INFO] --- maven-jar-plugin:2.4:jar (default-jar) @ ObsidianAutomation ---
[INFO]
[INFO] --- maven-failsafe-plugin:3.0.0-M3:integration-test (default) @ ObsidianAutomation ---
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running myTestRunners.MyTestRunner
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 0, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.052 s
[INFO] Finished at: 2021-07-31T18:11:52+02:00
[INFO] -----
```

Рисунок 3.3.2.Выполнение тестов в консоли

В ходе тестирования была выполнена проверка работы механизма параллельного выполнения TestNG. Использование параметров parallel="tests" и thread-count позволило одновременно запускать несколько

browser-session, что обеспечило сокращение общего времени выполнения тестового набора.

При последовательном запуске тестов выполнение полного набора сценариев занимало больше времени, поскольку каждый тест выполнялся отдельно и ожидал завершения предыдущего сценария. Использование параллельного выполнения позволило существенно повысить производительность тестирования за счёт одновременного запуска нескольких тестов.

Результаты сравнения времени выполнения представлены в таблице 3.3.2.

Таблица 3.3.2 — Сравнение времени выполнения тестов

Режим выполнения	Время выполнения
Последовательный запуск	~4 мин
Параллельный запуск	~1,5–2 мин

В процессе выполнения тестов также анализировалась стабильность работы разработанного фреймворка. Благодаря использованию ThreadLocal и независимых browser-session удалось избежать конфликтов WebDriver при многопоточном запуске тестов. Тестовые сценарии выполнялись независимо друг от друга и корректно завершались даже при одновременной работе нескольких потоков.

Дополнительно выполнялась проверка устойчивости инфраструктуры Selenium Grid при запуске тестов в контейнеризированной среде Docker. Использование контейнеров обеспечило воспроизводимость тестового окружения и позволило изолировать браузерные сессии, что положительно повлияло на стабильность выполнения UI-тестов.

Результаты проведённого анализа показали, что разработанный фреймворк обеспечивает корректное распределённое выполнение тестов,

поддержку параллельного запуска и стабильную работу тестовой инфраструктуры при выполнении автоматизированного UI-тестирования.

4. Выводы по третьей главе

3.4.1. Основные результаты реализации

В рамках третьей главы была выполнена практическая реализация фреймворка автоматизации UI-тестирования веб-приложений с использованием технологий Java, Selenium WebDriver, TestNG, Selenium Grid и Docker.

В процессе реализации была настроена структура проекта и подключены необходимые зависимости через систему сборки Maven. Дополнительно были реализованы базовые компоненты архитектуры фреймворка, включая классы BaseTest, BasePage и DriverFactory, обеспечивающие управление жизненным циклом тестов и экземплярами WebDriver.

Для организации взаимодействия с пользовательским интерфейсом веб-приложения был реализован подход Page Object Model, позволивший разделить тестовую логику и работу с элементами интерфейса. На основе данного подхода были разработаны Page Object компоненты для основных страниц тестируемого веб-приложения.

В рамках реализации распределённой инфраструктуры была выполнена настройка Selenium Grid и Docker-контейнеров, обеспечивающих удалённое и параллельное выполнение UI-тестов. Использование Selenium Grid совместно с TestNG позволило реализовать механизм параллельного запуска тестовых сценариев и сократить общее время тестирования.

Дополнительно была выполнена интеграция системы Allure Reports, обеспечивающей автоматическое формирование отчётности, отображение шагов тестирования, логирование ошибок и прикрепление скриншотов при возникновении сбоев.

Результаты тестирования показали, что разработанный фреймворк обеспечивает стабильное выполнение UI-тестов, поддержку распределённого

запуска и удобный механизм анализа результатов тестирования. Реализованное решение может использоваться как основа для дальнейшего расширения тестового проекта и интеграции с CI/CD-инфраструктурой.

Глава 4. Тестирование разработанного фреймворка автоматизации UI-тестирования

1. Организация и выполнение тестирования

4.1.1. Описание тестового окружения

Для проведения автоматизированного UI-тестирования было подготовлено распределённое тестовое окружение, построенное с использованием Selenium Grid и Docker. Данный подход позволил обеспечить удалённое выполнение тестов, изоляцию браузерных сессий и возможность параллельного запуска тестовых сценариев.

В качестве центра распределения тестов использовался Selenium Grid Hub, обеспечивающий управление browser-session и распределение тестовых задач между доступными browser-node. Browser-node функционировали внутри Docker-контейнеров, что обеспечило воспроизводимость тестовой среды и упростило запуск инфраструктуры на различных вычислительных системах.

В рамках тестирования использовались браузеры Google Chrome и Mozilla Firefox. Подключение тестов к Selenium Grid осуществлялось через RemoteWebDriver. Использование нескольких браузеров позволило проверить корректность работы тестируемого веб-приложения в условиях кроссбраузерного выполнения.

Запуск тестов выполнялся с использованием фреймворка TestNG. Конфигурация тестового запуска задавалась в файле testng.xml, содержащем параметры параллельного выполнения, количество потоков и перечень выполняемых тестовых классов. Для ускорения выполнения тестовых сценариев применялся режим parallel execution с использованием нескольких потоков выполнения.

Формирование отчётности осуществлялось с использованием Allure Reports. В процессе тестирования система сохраняла результаты выполнения

тестов, информацию об ошибках, логи и скриншоты пользовательского интерфейса при возникновении исключений.

В качестве тестируемого веб-приложения использовался учебный интернет-магазин SauceDemo, предназначенный для демонстрации и отработки сценариев автоматизированного UI-тестирования.

4.1.2. Выполнение автоматизированных UI-тестов

После подготовки тестового окружения был выполнен запуск автоматизированных UI-тестов разработанного фреймворка. Выполнение тестовых сценариев осуществлялось с использованием TestNG и Selenium Grid. Запуск производился как в последовательном, так и в параллельном режиме, что позволило оценить корректность работы распределённой инфраструктуры и механизмов многопоточности.

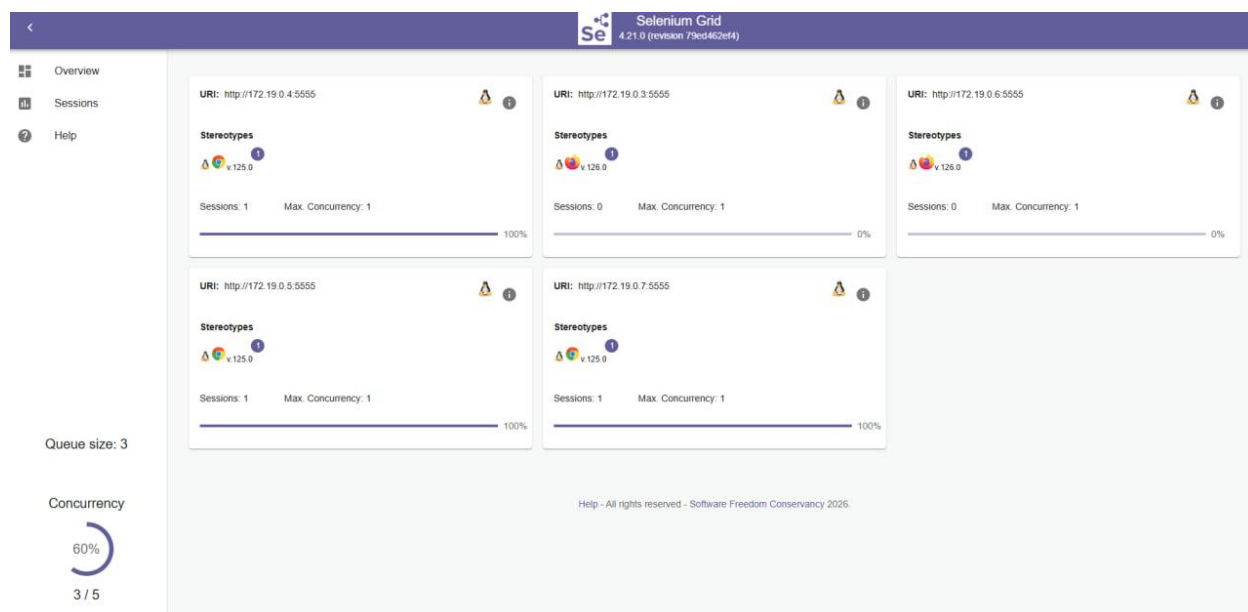


Рисунок 4.1.2. Выполнение тестов в Selenium Grid

В рамках тестирования были реализованы и выполнены тестовые сценарии для основных функциональных модулей веб-приложения SauceDemo. Класс LoginTest обеспечивал проверку сценариев авторизации пользователей, включая успешный вход, обработку некорректных учётных данных и отображение сообщений об ошибках. Класс ProductsTest выполнял проверки отображения каталога товаров, корректности отображения цен и названий товаров, а также тестирование механизмов сортировки. Класс

CartTest содержал сценарии добавления и удаления товаров из корзины, а также проверки корректности отображения счётчика товаров. В классе CheckoutTest были реализованы проверки процесса оформления заказа, включая заполнение формы, обработку обязательных полей и завершение оформления покупки.

Таблица 4.1.2. Тестовые классы и их назначения

Тестовый класс	Назначение
LoginTest	Проверка авторизации
ProductsTest	Проверка каталога товаров
CartTest	Проверка корзины
CheckoutTest	Проверка оформления заказа

Запуск тестов выполнялся через Selenium Grid с использованием RemoteWebDriver. Selenium Hub распределял browser-session между browser-node, функционирующими внутри Docker-контейнеров. В процессе тестирования использовались контейнеры с браузерами Google Chrome и Mozilla Firefox, что обеспечивало выполнение кроссбраузерного тестирования.

Для сокращения общего времени выполнения тестов использовался механизм parallel execution, настраиваемый через файл testng.xml. Выполнение нескольких тестовых сценариев одновременно позволило эффективно использовать ресурсы Selenium Grid и снизить продолжительность тестового запуска. При этом каждый поток выполнения использовал отдельную browser-session, что обеспечивало независимость тестов и предотвращало взаимное влияние сценариев друг на друга.

Результаты выполнения тестов отображались в консоли TestNG и дополнительно сохранялись в отчётах Allure Reports. В процессе выполнения тестов система автоматически фиксировала информацию о прохождении

тестов, возникающих ошибках и состоянии пользовательского интерфейса при возникновении исключений.

2. Анализ результатов тестирования

4.2.1. Анализ результатов выполнения тестов

После завершения выполнения автоматизированных UI-тестов был проведён анализ полученных результатов тестирования. Выполнение тестовых сценариев осуществлялось с использованием Selenium Grid и TestNG, а результаты тестовых запусков сохранялись в системе отчётности Allure Reports.

В процессе тестирования были успешно выполнены сценарии проверки авторизации пользователей, работы каталога товаров, корзины и оформления заказа. Разработанный фреймворк обеспечил корректное выполнение тестовых сценариев как в последовательном, так и в параллельном режиме. В ходе выполнения тестов Selenium Grid корректно распределял browser-session между доступными browser-node, функционирующими внутри Docker-контейнеров.

Таблица 4.2.1. Результаты выполнения тестовых сценариев

Тестовый модуль	Количество тестов	Результат
LoginTest	6	Успешно
ProductsTest	7	Успешно
CartTest	6	Успешно
CheckoutTest	6	Успешно

Формирование отчётности осуществлялось с использованием Allure Reports. Система автоматически сохраняла результаты выполнения тестов, информацию о статусе тестовых сценариев, времени выполнения, а также данные о возникших ошибках. Для каждого тестового сценария

отображались шаги выполнения тестов, что упрощало анализ результатов и локализацию дефектов.

При возникновении исключений фреймворк автоматически фиксировал состояние пользовательского интерфейса с помощью скриншотов и сохранял информацию об ошибках в логах тестирования. Это позволило упростить анализ причин возникновения ошибок и повысить удобство сопровождения тестового набора.

В ходе тестовых запусков была подтверждена стабильность работы разработанного фреймворка. При многократном выполнении тестов не наблюдалось критических ошибок, связанных с параллельным запуском или конфликтом `browser-session`. Использование `ThreadLocal` обеспечило независимость экземпляров `WebDriver` в каждом потоке выполнения, что позволило избежать взаимного влияния тестовых сценариев.

На рисунке 4.2.1 представлен пример сформированного отчёта Allure Reports. Отчёт содержит информацию о количестве успешно выполненных тестов, времени выполнения сценариев и статусе прохождения тестов.

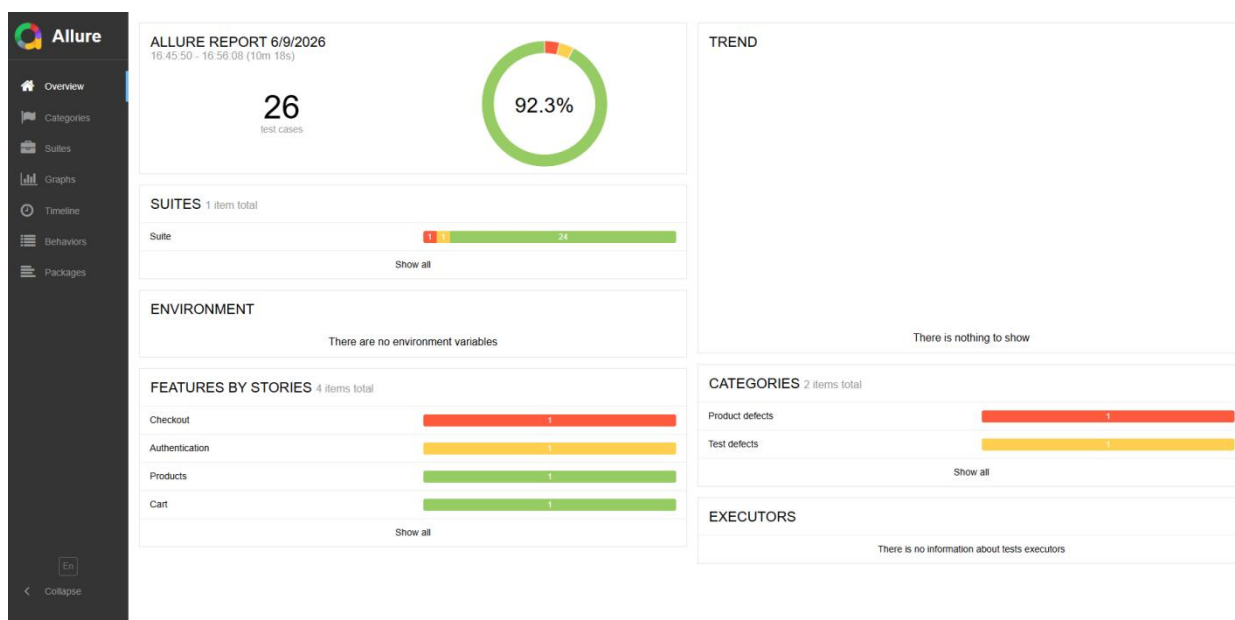


Рисунок 4.2.1.1. Главная страница отчёта Allure Reports

Также в рамках анализа результатов тестирования было установлено, что использование распределённой инфраструктуры Selenium Grid и механизмов `parallel execution` позволило повысить эффективность

выполнения тестового набора и сократить общее время тестирования по сравнению с последовательным запуском тестов.

```
[INFO]
[ERROR] Tests run: 28, Failures: 5, Errors: 0, Skipped: 5
[INFO]
[ERROR] There are test failures.

See D:\УлГТУ - работы\8 семестр\диплом\automationTestingUIFramework\target\surefire-reports for the individual test results.
See dump files (if any exist) [date].dump, [date]-jvmRun[N].dump and [date].dumpstream.
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 02:09 min
[INFO] Finished at: 2026-06-09T16:40:46+04:00
[INFO] -----
PS D:\УлГТУ - работы\8 семестр\диплом\automationTestingUIFramework> |
```

Рисунок 4.2.1.2. Результат выполнения тестов

3. Выводы по четвертой главе

4.3.1. Результаты тестирования разработанного фреймворка

В ходе тестирования была подтверждена работоспособность разработанного фреймворка автоматизации UI-тестирования веб-приложений. Реализованная архитектура обеспечила стабильное выполнение тестовых сценариев, поддержку распределённого запуска и формирование отчётности о результатах тестирования.

Проведённые тестовые запуски показали корректную работу UI-тестов для основных функциональных модулей веб-приложения, включая авторизацию пользователей, работу с каталогом товаров, корзиной и оформлением заказа. Использование паттерна Page Object Model позволило обеспечить поддерживаемость тестового кода и уменьшить дублирование компонентов.

Применение Selenium Grid и Docker обеспечило возможность распределённого выполнения тестов в различных браузерах. Использование контейнеризации позволило изолировать browser-session и обеспечить воспроизводимость тестового окружения. В процессе выполнения тестов

была подтверждена корректная работа механизмов удалённого запуска и распределения тестовых сценариев между browser-node.

Использование parallel execution позволило сократить общее время выполнения тестового набора по сравнению с последовательным запуском. Применение ThreadLocal обеспечило независимость browser-session при многопоточном выполнении тестов и повысило стабильность тестирования.

Интеграция Allure Reports обеспечила формирование структурированной отчётности, содержащей результаты выполнения тестов, информацию об ошибках, шаги тестирования и скриншоты пользовательского интерфейса. Полученные результаты подтверждают эффективность разработанного фреймворка и возможность его дальнейшего использования для автоматизации UI-тестирования веб-приложений.

Таблица 4.1.3. Сводка результатов

Показатель	Результат
Количество реализованных тестов	24
Поддержка браузеров	Chrome, Firefox
Поддержка Selenium Grid	Реализована
Поддержка Docker	Реализована
Параллельный запуск	Реализован
Генерация Allure Reports	Реализована
Результат выполнения тестов	Успешно

Список источников

1. Теория тестирования ПО простыми словами: от основ до практики. // Хабр URL: <https://habr.com/ru/articles/960202/> (дата обращения: 01.01.2026).
2. Знакомство с тестированием веб-приложений. // Хабр URL: <https://habr.com/ru/companies/ruvds/articles/676752/> (дата обращения: 03.01.2026).
3. CI/CD и Jenkins в современном тестировании // qarocks.ru. URL: <https://qarocks.ru/ci-cd-jenkins-testing/> (дата обращения: 05.01.2026)
4. Официальная документация Selenium // www.selenium. URL: <https://www.selenium.dev/documentation/webdriver/> (дата обращения: 05.01.2026)
5. Официальная документация Allure // allurereport.org. URL: <https://allurereport.org/docs/> (дата обращения: 05.01.2026)
6. Тестирование в Java. TestNG. // Хабр URL: <https://habr.com/ru/articles/121234/> (дата обращения: 06.01.2026).
7. Официальная документация JUnit // junit.org. URL: <https://docs.junit.org/6.0.2/overview.html> (дата обращения: 07.01.2026)
8. Как выполнять много UI-тестов параллельно, используя Selenium Grid. // Хабр URL: <https://habr.com/ru/companies/avito/articles/352208/> (дата обращения: 08.01.2026).
9. Как проводить тестирование с использованием Jenkins. // kurshub.ru URL: <https://kurshub.ru/journal/blog/kak-provodit-testirovanie-s-ispolzovaniem-jenkins/> (дата обращения: 08.01.2026).
10. Что такое автоматизированное тестирование. // selectel.ru URL: <https://selectel.ru/blog/test-automation/> (дата обращения: 09.01.2026).
11. Frontend Vs Backend in eCommerce: What's the Difference? // BrainSpate. URL: <https://brainspate.com/blog/frontend-vs-backend-in-ecommerce-unlocking-web-development-secrets/> (дата обращения: 09.01.2026).

12. Definition of Software Testing // ASTQB. URL: <https://astqb.org/what-is-software-testing/> (дата обращения: 09.01.2026).
13. Testing // ISTQB Glossary. URL: <https://istqb-glossary.page/testing/> (дата обращения: 09.01.2026).
14. What is Testing? // ASTQB. URL: <https://astqb.org/1-1-what-is-testing/> (дата обращения: 09.01.2026).
15. What is UI Testing? // BrowserStack. URL: <https://www.browserstack.com/guide/what-is-ui-testing> (дата обращения: 09.01.2026).
16. Cross Browser Testing Guide // BrowserStack. URL: <https://www.browserstack.com/guide/cross-browser-testing> (дата обращения: 09.01.2026).
17. Selenium Documentation // Selenium. URL: <https://www.selenium.dev/documentation/> (дата обращения: 09.01.2026).
18. 7. ISTQB® Certified Tester Foundation Level Syllabus Version 4.0 // ISTQB. URL: <https://istqb.org/certifications/certified-tester-foundation-level/> (дата обращения: 09.01.2026).
19. What is Test Automation? // BrowserStack. URL: <https://www.browserstack.com/guide/test-automation> (дата обращения: 09.01.2026).
20. Selenium Documentation // Selenium. URL: <https://www.selenium.dev/documentation/> (дата обращения: 09.01.2026).
21. Continuous Integration // Atlassian. URL: <https://www.atlassian.com/continuous-delivery/continuous-integration> (дата обращения: 09.01.2026).
22. ISTQB® Certified Tester Foundation Level Syllabus Version 4.0 // ISTQB. URL: <https://istqb.org/certifications/certified-tester-foundation-level/> (дата обращения: 09.01.2026).

23. Test Pyramid // Martin Fowler. URL: <https://martinfowler.com/articles/practical-test-pyramid.html> (дата обращения: 09.01.2026).
24. API Testing Guide // Postman Learning Center. URL: <https://learning.postman.com/docs/testing-and-validation/> (дата обращения: 09.01.2026).
25. Selenium Documentation // Selenium. URL: <https://www.selenium.dev/documentation/> (дата обращения: 09.01.2026).
26. TestNG Documentation // TestNG. URL: <https://testng.org/> (дата обращения: 09.01.2026).
27. JUnit 5 User Guide // JUnit. URL: <https://junit.org/junit5/docs/current/user-guide/> (дата обращения: 09.01.2026).
28. Postman Learning Center: Testing and Validation // Postman. URL: <https://learning.postman.com/docs/testing-and-validation/> (дата обращения: 09.01.2026).
29. Jenkins Documentation // Jenkins. URL: <https://www.jenkins.io/doc/> (дата обращения: 09.01.2026).
30. Allure Report Documentation // Allure. URL: <https://allurereport.org/docs/> (дата обращения: 09.01.2026).
31. Benefits and Limitations of Test Automation // BrowserStack. URL: <https://www.browserstack.com/guide/test-automation> (дата обращения: 09.01.2026).
32. Майерс Г. Дж., Сандлер К., Бэджетт Т. Искусство тестирования программ. 3-е изд. – М.: Вильямс, 2014. – 272 с.
33. Савин Р. Тестирование DOT COM, или Пособие по жестокому обращению с багами в интернет-стартапах. – М.: Дело, 2018. – 312 с.
34. Соммервилл И. Инженерия программного обеспечения. 10-е изд. – М.: Вильямс, 2019. – 848 с.
35. ГОСТ Р ИСО/МЭК 25010–2015. Системная и программная инженерия. Требования и оценка качества систем и программных продуктов

- (SQuaRE). Модели качества систем и программных продуктов. – М.: Стандартинформ, 2015.
- 36.ГОСТ Р ИСО/МЭК 12207–2010. Системная и программная инженерия. Процессы жизненного цикла программных средств. – М.: Стандартинформ, 2011.
- 37.Oracle Corporation. Java Documentation. URL: <https://docs.oracle.com/javase/> (дата обращения: 09.01.2026).
- 38.Apache Maven Project. Maven Documentation. URL: <https://maven.apache.org/guides/> (дата обращения: 09.01.2026).
- 39.Docker Documentation // Docker. URL: <https://docs.docker.com/> (дата обращения: 09.01.2026).
- 40.Docker Compose Documentation // Docker. URL: <https://docs.docker.com/compose/> (дата обращения: 09.01.2026).
- 41.Selenium Grid Documentation // Selenium. URL: <https://www.selenium.dev/documentation/grid/> (дата обращения: 09.01.2026).
- 42.Selenium WebDriver Documentation // Selenium. URL: <https://www.selenium.dev/documentation/webdriver/> (дата обращения: 09.01.2026).
- 43.Test Automation Pyramid // Martin Fowler. URL: <https://martinfowler.com/articles/practical-test-pyramid.html> (дата обращения: 09.01.2026).
- 44.3BrowserStack. Guide to Selenium Automation Testing. URL: <https://www.browserstack.com/guide/selenium-automation-testing> (дата обращения: 09.01.2026).
- 45.Selectel. Что такое автоматизированное тестирование. URL: <https://selectel.ru/blog/test-automation/> (дата обращения: 09.01.2026).
- 46.Хабр. Автоматизация тестирования: основные подходы и инструменты. URL: <https://habr.com/ru/companies/otus/articles/> (дата обращения: 09.01.2026).

Приложение А. Исходный код класса инициализации драйвера для взаимодействия с браузером, DriverFactory.java.

```
package driver;

import io.github.bonigarcia.wdm.WebDriverManager;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.firefox.FirefoxDriver;
import utils.ConfigReader;

import org.openqa.selenium.remote.RemoteWebDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.firefox.FirefoxOptions;

import java.net.URL;

import java.time.Duration;

public class DriverFactory {

    private static final ThreadLocal<WebDriver> driverThreadLocal = new ThreadLocal<>();

    public static WebDriver getDriver() {

        if (driverThreadLocal.get() == null) {

            String browser = ConfigReader.getProperty("browser");
            String execution = ConfigReader.getProperty("execution");

            try {

                WebDriver driver;

                if (execution.equalsIgnoreCase("remote")) {

                    String gridUrl = ConfigReader.getProperty("gridUrl");

                    switch (browser.toLowerCase()) {
                        case "chrome":
                            driver = new RemoteWebDriver(
                                new URL(gridUrl),
                                new ChromeOptions()
                            );
                            break;

                        case "firefox":
                            driver = new RemoteWebDriver(
                                new URL(gridUrl),
                                new FirefoxOptions()
                            );
                            break;

                        default:
                            throw new RuntimeException("Unsupported browser");
                    }
                }
            }
        }
    }
}
```

```

    } else {
        // локальный запуск (как у тебя уже есть)
        switch (browser.toLowerCase()){
            case "chrome":
                WebDriverManager.chromedriver().setup();
                driver = new ChromeDriver();
                break;

            case "firefox":
                WebDriverManager.firefoxdriver().setup();
                driver = new FirefoxDriver();
                break;

            default: throw new RuntimeException("Unsupported browser");
        }
    }

    driver.manage().window().maximize();
    driver.manage().timeouts().implicitlyWait(
        Duration.ofSeconds(
            Integer.parseInt(ConfigReader.getProperty("implicitWait"))
        )
    );

    driverThreadLocal.set(driver);

    } catch (Exception e) {
        throw new RuntimeException("Failed to create driver", e);
    }
}

return driverThreadLocal.get();
}

public static void quitDriver() {
    if(driverThreadLocal.get() != null) {
        driverThreadLocal.get().quit();
        driverThreadLocal.remove();
    }
}
}
}

```

Приложение Б. Исходный код файла конфигурации фреймворка, pom.xml.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>org.example</groupId>
  <artifactId>automationTestingUIFramework</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>19</maven.compiler.source>
    <maven.compiler.target>19</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <aspectj.version>1.9.22</aspectj.version>
    <allure.version>2.33.0</allure.version>
    <selenium.version>4.33.0</selenium.version>
  </properties>

  <dependencies>

    <!-- Source: https://mvnrepository.com/artifact/io.github.bonigarcia/webdrivermanager -->
    <dependency>
      <groupId>io.github.bonigarcia</groupId>
      <artifactId>webdrivermanager</artifactId>
      <version>6.3.3</version>
      <scope>compile</scope>
    </dependency>

    <!-- Source: https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java -->
    <dependency>
      <groupId>org.seleniumhq.selenium</groupId>
      <artifactId>selenium-java</artifactId>
      <version>4.41.0</version>
      <scope>compile</scope>
    </dependency>

    <!-- Source: https://mvnrepository.com/artifact/org.testng/testng -->
    <dependency>
```

```

    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>7.12.0</version>
    <scope>test</scope>
</dependency>

<!-- Source: https://mvnrepository.com/artifact/io.qameta.allure/allure-testng -->
<dependency>
    <groupId>io.qameta.allure</groupId>
    <artifactId>allure-testng</artifactId>
    <version>2.33.0</version>
    <scope>test</scope>
</dependency>

</dependencies>

<build>
    <plugins>
        <!-- Maven Surefire Plugin -->
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-surefire-plugin</artifactId>
            <version>3.5.2</version>
            <configuration>
                <suiteXmlFiles>
                    <suiteXmlFile>src/test/resources/testng.xml</suiteXmlFile>
                </suiteXmlFiles>
                <testFailureIgnore>true</testFailureIgnore>
            </configuration>
            <dependencies>
                <dependency>
                    <groupId>org.aspectj</groupId>
                    <artifactId>aspectjweaver</artifactId>
                    <version>1.9.22</version>
                </dependency>
            </dependencies>
        </plugin>

        <!-- Allure Maven Plugin -->
        <plugin>
            <groupId>io.qameta.allure</groupId>
            <artifactId>allure-maven</artifactId>

```

```
<version>2.15.2</version>
<configuration>
  <reportVersion>${allure.version}</reportVersion>
  <resultsDirectory>${project.build.directory}/allure-results</resultsDirectory>
  <reportDirectory>${project.build.directory}/allure-report</reportDirectory>
</configuration>
</plugin>
</plugins>
</build>
</project>
```

Приложение В. Исходный код класса для запуска тестов страницы авторизации, LoginTest.java.

```
package tests;

import core.BaseTest;
import io.qameta.allure.*;
import org.testng.Assert;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;
import pages.InventoryPage;
import pages.LoginPage;

@Epic("Authentication")
@Feature("Login")
public class LoginTest extends BaseTest {

    private LoginPage loginPage;
    private InventoryPage inventoryPage;

    @BeforeMethod
    public void initPages() {
        loginPage = new LoginPage(driver);
        inventoryPage = new InventoryPage(driver);
    }

    @Test
    @Story("Successful login")
    @Severity(SeverityLevel.CRITICAL)
    @Description("Проверка успешной авторизации с валидными данными")
    public void testSuccessfulLogin() {
        loginPage.
            login(
                "standard_user",
                "secret_sauce"
            );
        Assert.assertTrue(
            inventoryPage.isOpened(),
            "Inventory page is not opened"
        );
    }

    @Test
    @Story("Login with invalid password")
    @Severity(SeverityLevel.CRITICAL)
    @Description("Попытка входа с неверным паролем")
    public void testLoginWithInvalidPassword() {
        loginPage
            .login(
                "standard_user",
                ""
            );
        Assert.assertTrue(
            loginPage
                .getErrorMessage()
                .contains("Epic sadface: Password is required")
        );
    }

    @Test
```

```

@Story("Login with invalid username")
@Severity(SeverityLevel.CRITICAL)
@Description("Попытка входа с неверным логином")
public void testLoginWithInvalidUsername() {
    loginPage.login(
        "wrong_user",
        "secret_sauce"
    );
    Assert.assertTrue(
        loginPage
            .getErrorMessage()
            .contains("Username and password do not match")
    );
}

@Test
@Story("Login with empty fields")
@Severity(SeverityLevel.NORMAL)
@Description("Попытка входа с пустыми полями")
public void testLoginWithEmptyFields() {
    loginPage.clickLogin();
    Assert.assertTrue(
        loginPage
            .getErrorMessage()
            .contains("Username is required")
    );
}

@Test
@Story("Error message validation")
@Severity(SeverityLevel.NORMAL)
@Description("Проверка точного текста ошибки")
public void testErrorMessageText() {
    loginPage
        .login(
            "wrong_user",
            "wrong_password"
        );
    Assert.assertEquals(
        loginPage.getErrorMessage(),
        "Epic sadface: Username and password do not match any user in this service"
    );
}

@Test
@Story("Locked user login")
@Severity(SeverityLevel.CRITICAL)
@Description("Попытка входа заблокированного пользователя")
public void testLoginWithLockedUser() {
    loginPage
        .login(
            "locked_out_user",
            "secret_sauce"
        );
    Assert.assertTrue(
        loginPage
            .getErrorMessage()
            .contains("Epic sadface: Sorry, this user has been locked out.")
    );
}
}

```

Приложение Г. Исходный код класса абстракции страницы авторизации, **LoginPage.java.**

```
package pages;

import core.BasePage;
import io.qameta.allure.Step;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;

public class LoginPage extends BasePage {

    public LoginPage(WebDriver driver) {
        super(driver);
    }

    @FindBy(id = "user-name")
    private WebElement usernameInput;

    @FindBy(id = "password")
    private WebElement passwordInput;

    @FindBy(id = "login-button")
    private WebElement loginButton;

    @FindBy(css = "[data-test='error']")
    private WebElement errorMessage;

    @Step("Ввод логина: {username}")
    public void enterUsername(String username) {
        usernameInput.sendKeys(username);
    }

    @Step("Ввод пароля")
    public void enterPassword(String password) {
        passwordInput.sendKeys(password);
    }

    @Step("Нажатие кнопки Login")
    public void clickLogin() {
        loginButton.click();
    }

    @Step("Логин как пользователь: {username}")
    public void login(String username, String password) {
        enterUsername(username);
        enterPassword(password);
        clickLogin();
    }

    @Step("Получение текста ошибки")
    public String getErrorMessage() {
        return errorMessage.getText();
    }
}
```

**Приложение Д. Ссылка на репозиторий гитхаб с исходным кодом
фреймворка.**

https://github.com/kaban73/diplom_work